Rapport de projet

ProSys – Odoo 14 en haute disponibilité sur LXC

Architecture, déploiement Ansible et exploitation

ADMIN

Résumé

Ce rapport décrit le projet ProSys : le déploiement par Ansible de trois instances **Odoo 14** qui partagent une base **PostgreSQL 15** rendue hautement disponible par **Patroni**, **etcd** et **HAProxy**, et un filestore commun porté par un volume **GlusterFS**. Chaque instance tourne dans un conteneur LXC Debian 12, posé sur son propre hôte et relié au réseau en IPVLAN.

Le rapport présente l'architecture, les mécanismes de haute disponibilité (réplication, bascule automatique et planifiée, quorum, réintégration), la sécurité des flux, la sauvegarde, l'installation et l'exploitation, puis détaille chacun des huit playbooks. Les vingt-quatre schémas sont suivis chacun d'une lecture commentée : quatorze décrivent l'architecture actuelle, dix proviennent des éditions précédentes et montrent son évolution. Les limites connues et les points restant à compléter sont signalés.

Table des matières

1 Introduction **3**
1.1 Contexte et objectifs . . . 3
1.2 Principes SRE appliqués . . . 3

2 Architecture générale **4**
2.1 Vue d'ensemble . . . 4
2.2 Machines et conteneurs . . . 5
2.3 Filestore commun . . . 6

3 Haute disponibilité de PostgreSQL **7**
3.1 Rôle des composants . . . 7
3.2 Connexion d'Odoo à la base . . . 8
3.3 Réplication . . . 9
3.4 Bascule automatique . . . 11
3.5 Quorum et partitions réseau . . . 12
3.6 Bascule planifiée . . . 13

4 Réseau et sécurité des flux **14**

5 Installation et déploiement d'Odoo **15**
5.1 Prérequis du poste de contrôle . . . 15
5.2 Ordre de déploiement . . . 15
5.3 Pile d'exécution d'Odoo . . . 17

6 Les playbooks en détail **19**
6.1 `ansi2-snapchot.yaml` – créer les conteneurs . . . 19
6.2 `reseau-lxc.yaml` – relier les conteneurs au réseau . . . 20
6.3 `ansi_odoo.yaml` – déploiement complet . . . 20
6.4 `ansi_postgresql_ha.yaml` – haute disponibilité de la base . . . 22
6.5 `nettoyage_odoo.yaml` – réparer un conteneur Odoo . . . 23
6.6 `nettoyage.yaml` – tout désinstaller . . . 23

7 Vérification et exploitation **23**
7.1 Vérification après déploiement . . . 23
7.2 Exploitation courante . . . 24

8 Évolution de l'architecture **24**
8.1 Réseau des conteneurs LXC 24
8.2 Filestore GlusterFS 27
8.3 Architecture et HA PostgreSQL 30

9 Limites connues **34**

10 Conclusion **35**

A Catalogue des schémas **35**

1 Introduction

1.1 Contexte et objectifs

ProSys exploite Odoo 14 pour plusieurs utilisateurs. Le projet répond à trois objectifs :

— **servir une base unique** à trois instances Odoo, pour que toutes voient les mêmes données ;

— **rendre la base hautement disponible** : si le serveur PostgreSQL primaire tombe, une réserve prend sa place sans intervention ;

— **rendre le déploiement reproductible** : toute l'installation est décrite par des playbooks Ansible versionnés.

1.2 Principes SRE appliqués

L'ingénierie de la fiabilité (SRE, *Site Reliability Engineering*) traite l'exploitation comme un problème d'ingénierie : la fiabilité se définit et se mesure, le travail répétitif s'automatise, les changements sont rendus sûrs, et chaque incident sert à renforcer le système. Le projet applique ces principes de la façon suivante ; les points encore ouverts sont signalés.

— **Supprimer les points de défaillance uniques.** Trois serveurs PostgreSQL sous Patroni, un quorum etcd de 2 sur 3, un HAProxy par serveur et trois instances Odoo : la perte d'un nœud de données est tolérée sans intervention. Restent uniques : le cron d'Odoo, porté par odoo1 ; le serveur 192.168.36.8, dont dépend le montage du volume au démarrage, aucun serveur de secours n'étant déclaré ; l'hôte de sauvegarde.

— **Automatiser plutôt que répéter** (réduction du *toil*). Tout le déploiement est décrit par des playbooks Ansible versionnés. Ils sont idempotents : rejoués, ils remettent en conformité sans rien casser, et `--check --diff` montre ce qui changerait avant d'agir.

— **Rendre les changements sûrs.** Les opérations risquées sont progressives, pour limiter le rayon d'impact : montage du filestore un hôte à la fois, relance de Patroni un serveur à la fois et cluster en pause, redémarrages des réserves avant le primaire. À l'inverse, une reprise à moitié faite étant pire qu'une reprise pas commencée, un échec sur un serveur PostgreSQL arrête les trois (`any_errors_fatal`).

— **Échouer de façon sûre** (garde-fous). Odoo ne démarre pas sans son volume partagé, et le rôle `odoo14` refuse de configurer une instance sans lui. Le rôle `pg_ha` refuse de créer une nouvelle autorité de certification et s'arrête sur « refus » avant toute modification. La base n'est initialisée que par odoo1.

— **Apprendre des incidents.** Les incidents rencontrés sont documentés dans le code, avec leur date et leur cause, et chacun a donné un garde-fou : sauvegarde refusée par la base faute de règle pour son hôte (24/09/2026), Odoo écrivant sur son disque local faute de volume (25/09/2026), instances arrêtées par un mauvais chemin du filestore dans le volume (29 et 30/09/2026), base voyant les Odoo sous l'adresse du routeur NAT (20/10/2026). C'est l'esprit du post-mortem sans recherche de coupable : corriger le système, pas les personnes.

— **Rendre l'environnement reproductible.** Les versions des collections Ansible sont épinglées, et un contrôle optionnel (tag `preflight`) les vérifie avant tout contact avec les hôtes. L'archive de Python 3.8.20 est contrôlée par son empreinte SHA256 ; les dépendances Python viennent d'un fichier de versions figées lorsqu'il existe.

— **Prévoir la capacité.** La règle du projet borne les connexions à la base : 3 nœuds × (2 workers + 1 cron) × 16 connexions, soit 144 au plus, pour 200 autorisées par `max_connections`.

— **Observer et vérifier.** Chaque déploiement se termine par des contrôles actifs : un primaire et deux réserves, retard inférieur à 16 Mo, vraie connexion TLS à travers chaque HAProxy, montage GlusterFS dans chaque conteneur. L'état se lit par `patronictl list`, l'API Patroni, la page d'état HAProxy et les journaux systemd. *À compléter :* le dépôt ne déploie ni supervision continue ni alertes.

— **Mesurer la fiabilité** (SLI, SLO, budget d'erreur). *À définir :* le dépôt ne fixe pas encore d'objectifs chiffrés. Indicateurs proposés : disponibilité de `/web/login` sur chaque instance, disponibilité des écritures SQL, temps de rétablissement après une bascule (RTO) et perte de données admise (RPO). Les réglages comme `ttl=30 s` ou `maximum_lag_on_failover` ne sont pas des mesures : seuls des tests de bascule et de restauration chronométrés les fourniront.

Le rapport décrit la configuration prévue par le code ; l'état réel des serveurs, les délais de bascule et la restauration restent à constater en exploitation.

2 Architecture générale

2.1 Vue d'ensemble

L'architecture sépare une zone applicative, où tournent les trois Odoo, et une zone de données, où tournent les trois serveurs PostgreSQL. Le filestore est porté par un volume GlusterFS extérieur au projet.

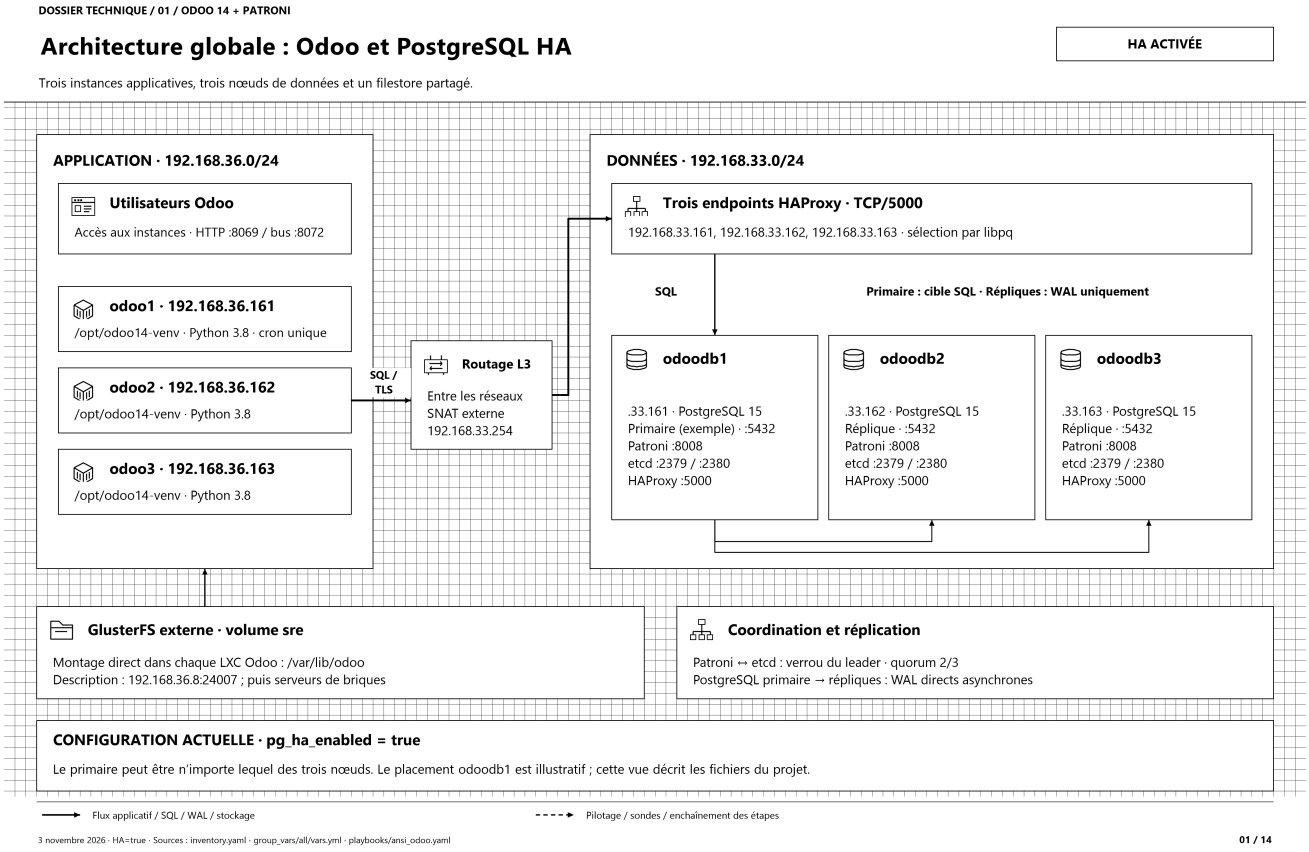


Schéma 1 – Architecture globale : Odoo et PostgreSQL HA

Lecture du schéma 1

Trois instances applicatives, trois nœuds de données et un filestore partagé.

- **Zone application** (`192.168.36.0/24`) : les utilisateurs accèdent directement à l'une des trois instances Odoo, en HTTP sur 8069 et sur 8072 pour le bus de notifications. Les trois instances tournent dans `/opt/odoo14-venv` avec Python 3.8 ; odoo1 porte en plus l'unique cron.
- **Routage L3** : un routeur externe relie les deux réseaux et applique un SNAT vers `192.168.33.254`.
- **Zone données** (`192.168.33.0/24`) : chaque nœud porte PostgreSQL 15 (5432), Patroni (8008), etcd (2379 et 2380) et HAProxy (5000).
- **Accès SQL** : Odoo connaît les trois points d'entrée HAProxy sur TCP/5000 ; libpq choisit l'adresse disponible, et le trafic SQL est chiffré en TLS.
- **Rôles** : seul le primaire reçoit le SQL. Les répliques ne reçoivent que les WAL, le journal des transactions du primaire (expliqué à la section 3.3), de façon asynchrone. Patroni et etcd coordonnent le verrou du leader avec un quorum de 2 sur 3.
- **Filestore** : le volume GlusterFS externe `sre` est monté directement dans chaque conteneur Odoo, sur `/var/lib/odoo`. `192.168.36.8:24007` fournit la description du volume, puis le client dialogue avec les serveurs de briques.
- Le placement « odoodb1 primaire » n'est qu'un exemple : le primaire peut être n'importe lequel des trois nœuds.

2.2 Machines et conteneurs

Chaque hôte Debian 12 porte un seul conteneur LXC. Le placement est défini dans `inventory.yaml` et dans `host_vars/`.

Rôle	Conteneur	IP du conteneur	Hôte	Carte
odoo1 – Odoo, initialisation de la base, cron	`devodoo14`	192.168.36.161	192.168.36.21	`enp5s0`
odoo2 – Odoo	`devodoo14`	192.168.36.162	192.168.36.22	`enp5s0`
odoo3 – Odoo	`devodoo14`	192.168.36.163	192.168.36.23	`enp5s0`
odoodb1 – PostgreSQL, Patroni, etcd, HAProxy	`devpg15`	192.168.33.161	192.168.33.21	`enp1s0`
odoodb2 – PostgreSQL, Patroni, etcd, HAProxy	`devpg15`	192.168.33.162	192.168.33.22	`enp1s0`
odoodb3 – PostgreSQL, Patroni, etcd, HAProxy	`devpg15`	192.168.33.163	192.168.33.23	`enp1s0`

Tableau 1 – Hôtes et conteneurs déclarés dans l'inventaire

Élément extérieur	Adresse	Usage
Volume GlusterFS répliqué `sre`	192.168.36.8 (description du volume)	Filestore et sessions communs aux trois Odoo
Routeur entre les deux réseaux	passerelles `.254`	NAT : la base voit les Odoo sous 192.168.33.254
Hôte de sauvegarde	192.168.36.21 (`backup_host`)	`pg_dump` et archive du filestore dans `/srv/sauvegardes`

Tableau 2 – Éléments extérieurs que le déploiement suppose en place

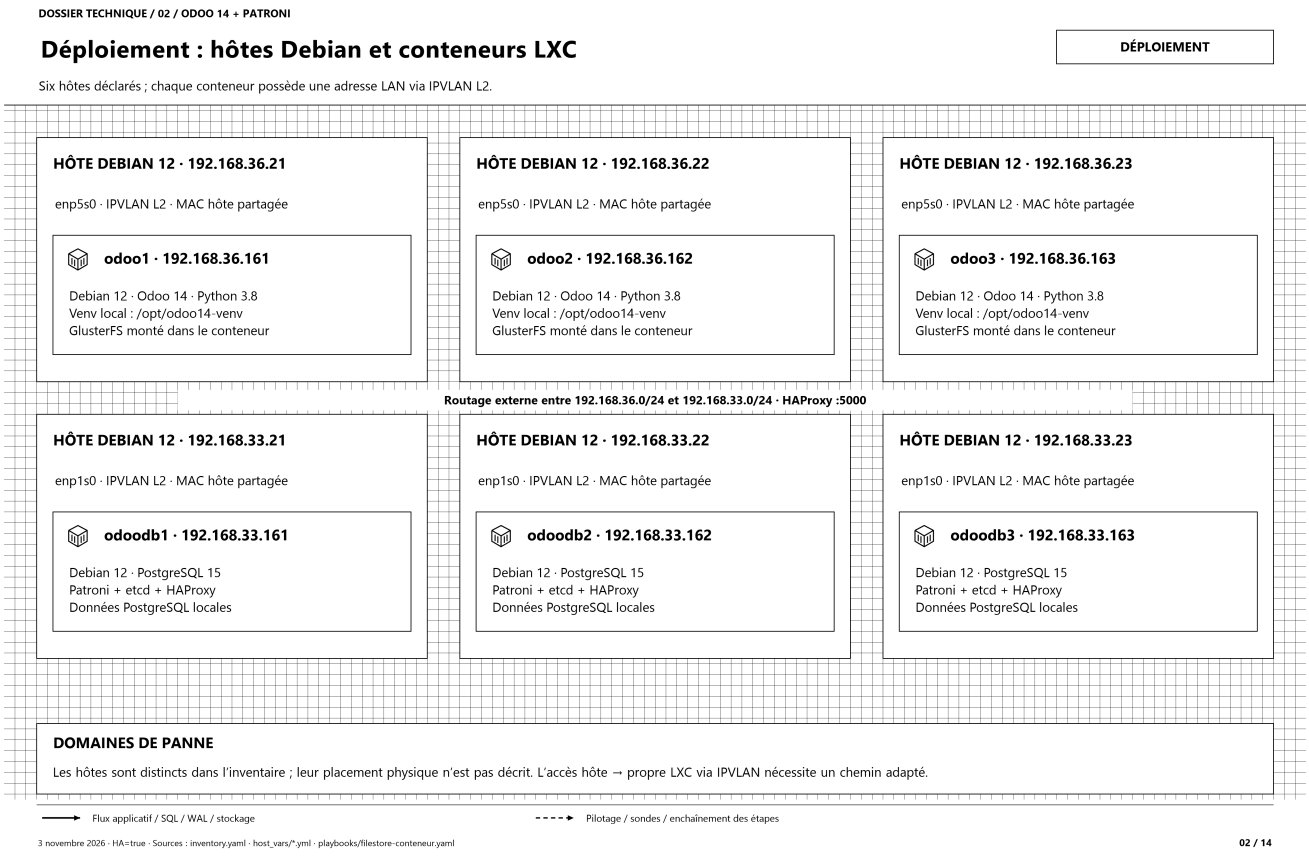


Schéma 2 – Déploiement : hôtes Debian et conteneurs LXC

Lecture du schéma 2

Six hôtes déclarés ; chaque conteneur possède une adresse LAN via IPVLAN L2.

- Chaque boîte représente un hôte Debian 12 et le conteneur qu'il porte. Les hôtes 192.168.36.21 à 23 utilisent la carte `enp5s0` et portent odoo1 à odoo3. Les hôtes 192.168.33.21 à 23 utilisent `enp1s0` et portent odoodb1 à odoodb3.
- Les conteneurs sont reliés en **IPVLAN L2** : ils partagent l'adresse MAC de leur hôte, la seule que le réseau accepte.
- **Conteneurs Odoo** : Debian 12, Odoo 14, Python 3.8, venv local `/opt/odoo14-venv`, volume GlusterFS monté dans le conteneur.
- **Conteneurs PostgreSQL** : PostgreSQL 15 avec Patroni, etcd et HAProxy. Les données PostgreSQL sont locales à chaque nœud et recopiées par réplication.
- Le routage externe relie 192.168.36.0/24 et 192.168.33.0/24 ; Odoo atteint les HAProxy sur le port 5000.
- **Domaines de panne** : les hôtes sont distincts dans l'inventaire, mais leur placement physique n'est pas décrit. En IPVLAN, un hôte ne joint pas son propre conteneur sans chemin adapté.

2.3 Filestore commun

Chaque conteneur Odoo monte *lui-même* `192.168.36.8:/sre` sur `/var/lib/odoo`. Le volume contient `filestore/prod` et `sessions`. Trois garde-fous protègent ce montage :

— l'unité systemd d'Odoo exige le montage (`RequiresMountsFor`). Sans volume, Odoo ne démarre pas, au lieu d'écrire sur le disque local ;

— le rôle `odoo14` refuse de configurer une instance dont `/var/lib/odoo` n'est pas un montage GlusterFS ;

— le compte `odoo` a le même UID et le même GID (105/112) sur les trois conteneurs. Sur un volume partagé, ce sont les numéros qui comptent.

L'ancien montage sur l'hôte suivi d'un bind vers le conteneur n'existe plus. Seul l'hôte de sauvegarde garde `/mnt/odoo`, en lecture seule.

3 Haute disponibilité de PostgreSQL

3.1 Rôle des composants

La haute disponibilité repose sur trois composants installés dans chacun des trois conteneurs PostgreSQL. Patroni pilote le PostgreSQL local, etcd conserve l'état du cluster, et HAProxy dirige les connexions vers le primaire.

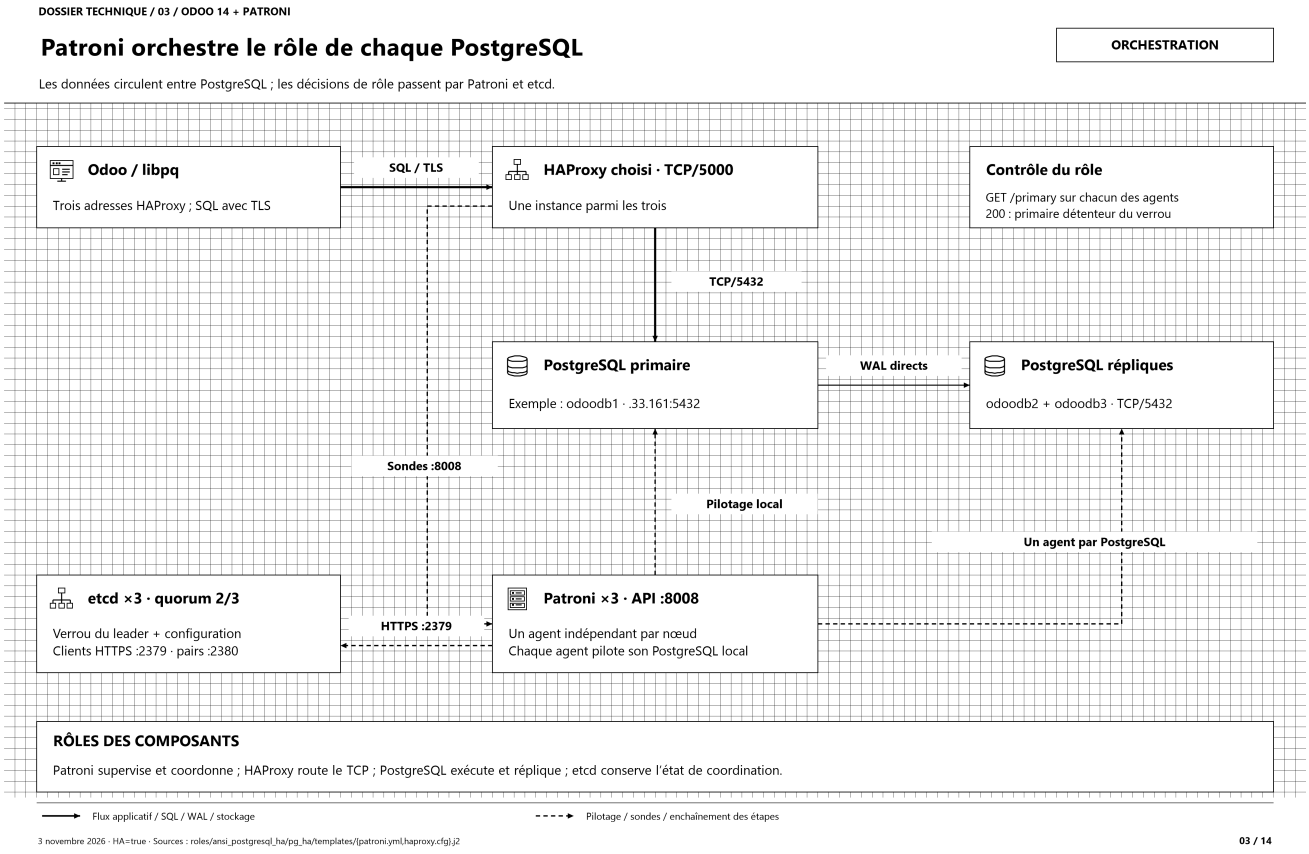


Schéma 3 – Patroni orchestre le rôle de chaque PostgreSQL

Lecture du schéma 3

Les données circulent entre PostgreSQL ; les décisions de rôle passent par Patroni et etcd.

— Odoo, par libpq, connaît trois adresses HAProxy. Le SQL chiffré passe par l'un des trois HAProxy (port 5000), puis rejoint le primaire sur 5432.

— HAProxy contrôle le rôle de chaque serveur par `GET /primary` auprès des trois agents Patroni : la réponse 200 désigne le primaire qui détient le verrou.

— Les WAL vont directement du primaire vers les répliques ; ils ne traversent pas Patroni.

— **Patroni** (API 8008) : un agent indépendant par nœud, qui démarre, arrête, configure et promeut son PostgreSQL local.

— **etcd** (quorum 2/3) : conserve le verrou du leader et la configuration ; clients en HTTPS sur 2379, échanges entre membres sur 2380.

— En résumé : Patroni supervise et coordonne, HAProxy route le TCP, PostgreSQL exécute et réplique, etcd conserve l'état de coordination.

3.2 Connexion d'Odoo à la base

Odoo reçoit dans `/etc/odoo/odoo.conf` la liste des trois serveurs et le port des HAProxy :

```
db_host = 192.168.33.161,192.168.33.162,192.168.33.163
db_port = 5000
```

Comme le routeur fait du NAT, la base reconnaît chaque instance à son **certificat client** et non à son adresse.

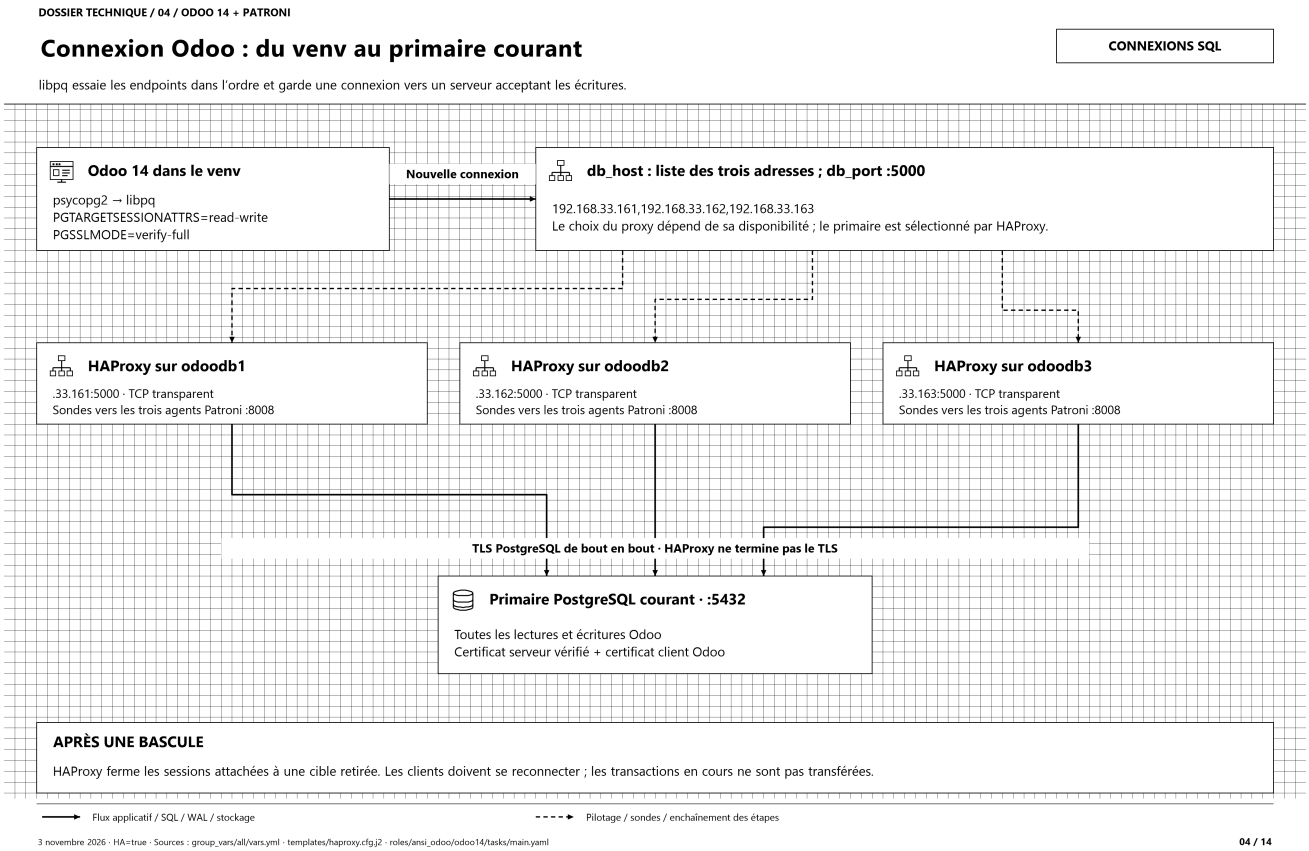


Schéma 4 – Connexion Odoo : du venv au primaire courant

Lecture du schéma 4

libpq essaie les points d'entrée dans l'ordre et garde une connexion vers un serveur qui accepte les écritures.

- Odoo 14 s'exécute dans le venv et passe par psycopg2 puis libpq, avec `PGTARGETSESSIONATTRS=read-write` et `PGSSLMODE=verify-full`.
- libpq essaie les trois adresses, port 5000, et ouvre une nouvelle connexion vers la première disponible. Le choix du proxy dépend de sa disponibilité ; c'est HAProxy qui sélectionne le primaire.
- Chaque HAProxy (sur odoodb1, odoodb2 et odoodb3) est un relais TCP transparent qui sonde les trois agents Patroni sur 8008.
- Toutes les connexions aboutissent au **primaire courant** (5432), qui reçoit toutes les lectures et écritures d'Odoo. Le certificat du serveur est vérifié, et celui d'Odoo est exigé.
- Le TLS PostgreSQL va de bout en bout : HAProxy ne le termine pas.
- Après une bascule, HAProxy ferme les sessions attachées à la cible retirée. Les clients doivent se reconnecter, et les transactions en cours ne sont pas transférées.

3.3 Réplication

Qu'est-ce que le WAL ? Le WAL (*Write-Ahead Log*, « journal écrit en premier ») est le journal des transactions de PostgreSQL. Avant de modifier ses fichiers de données, PostgreSQL écrit chaque changement dans ce journal, sous forme d'enregistrements numérotés à la suite, et le force sur le disque au moment du `COMMIT`. Le journal est découpé en fichiers de 16 Mo, rangés dans le répertoire `pg_wal/`. On parle couramment « des WAL » pour désigner ces fichiers ou les enregistrements qu'ils contiennent.

Ce principe sert à trois choses dans le projet :

- **reprise après une panne** : au redémarrage, PostgreSQL rejoue le WAL pour retrouver un état cohérent, même si les fichiers de données n'avaient pas encore été mis à jour ;
- **réplication** : le primaire envoie son WAL en continu (*streaming*) aux répliques, qui le rejouent. Elles obtiennent ainsi une copie exacte de la base. Ce ne sont pas les requêtes SQL qui sont transmises, mais les modifications physiques des pages de données ;
- **réintégration d'un ancien primaire** : `pg_rewind` s'appuie sur l'historique du WAL pour ne recopier que les blocs qui ont divergé. C'est pour cela que `wal_log_hints = on` est activé.

Quatre notions reviennent dans la suite :

- `wal_level = replica` : le WAL contient assez d'informations pour alimenter des répliques ;
- **slot de réplication** : le primaire conserve le WAL qu'une réplique n'a pas encore reçu. `max_slot_wal_keep_size = 2GB` borne cette réserve, pour qu'une réplique absente ne remplisse pas le disque du primaire ;
- **retard** (*lag*) : la quantité de WAL produite par le primaire et pas encore reçue ou rejouée par une réplique, mesurée en octets. C'est elle que comparent `maximum_lag_on_failover` (1 Mio) et le contrôle final d'Ansible (16 Mo) ;
- **asynchrone** : le primaire confirme le `COMMIT` à Odoo sans attendre qu'une réplique ait reçu le WAL correspondant. C'est rapide, mais les toutes dernières transactions peuvent manquer sur une réplique promue après une panne.

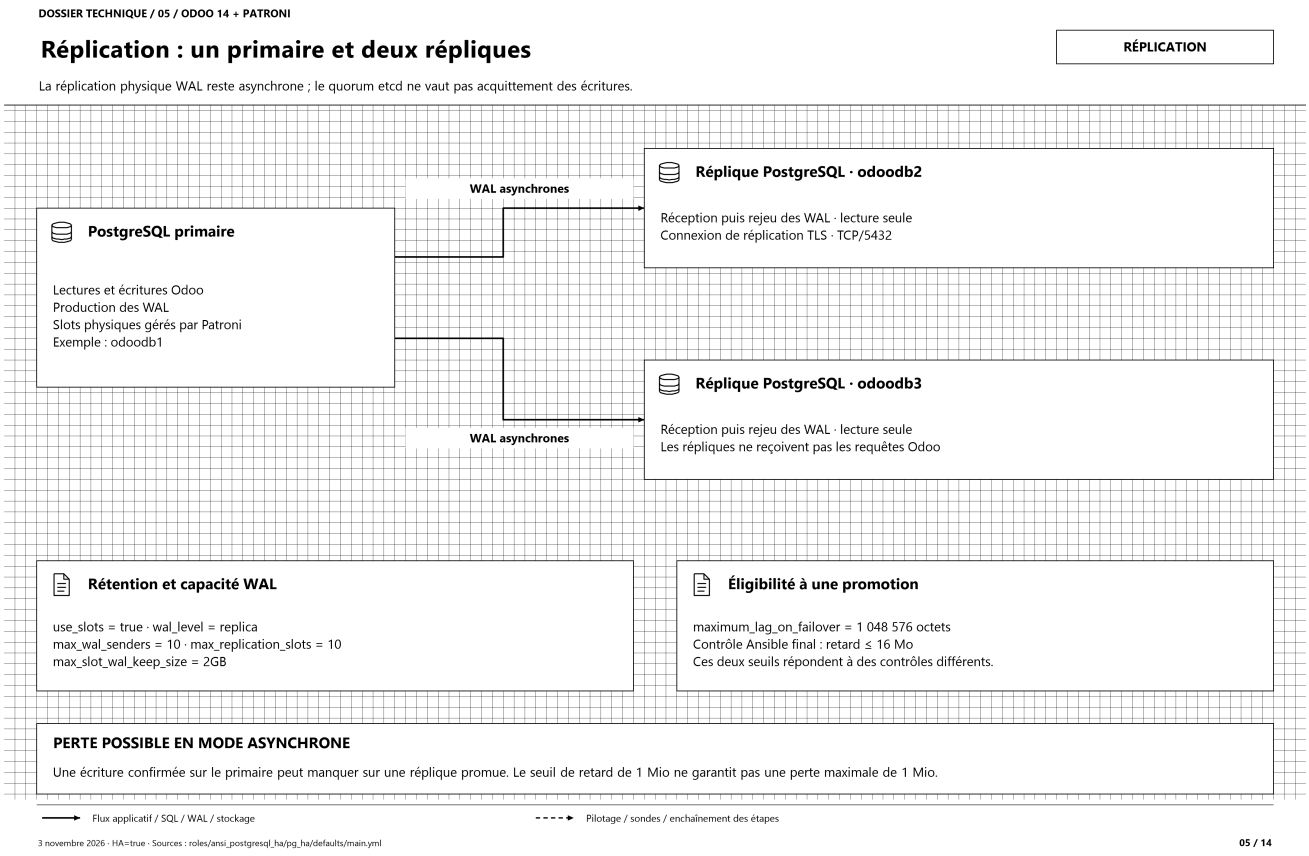


Schéma 5 – Réplication : un primaire et deux répliques

Lecture du schéma 5

La réplication physique des WAL reste asynchrone ; le quorum etcd ne vaut pas acquittement des écritures.

- Le **primaire** (exemple : odoodb1) traite les lectures et écritures d'Odoo, produit les WAL et utilise des slots physiques gérés par Patroni.
- Les **répliques** odoodb2 et odoodb3 reçoivent puis rejouent les WAL, en lecture seule, par une connexion de réplication TLS sur 5432. Elles ne reçoivent aucune requête d'Odoo.
- **Rétention** : `use_slots = true`, `wal_level = replica`, `max_wal_senders = 10`, `max_replication_slots = 10`, `max_slot_wal_keep_size = 2GB`.
- **Éligibilité** : `maximum_lag_on_failover` vaut 1 048 576 octets (1 Mio). Le contrôle final d'Ansible exige un retard inférieur ou égal à 16 Mo. Les deux seuils répondent à des contrôles différents.
- **Perte possible** : une écriture confirmée sur le primaire peut manquer sur une réplique promue. Le seuil de 1 Mio ne garantit pas une perte maximale de 1 Mio.

3.4 Bascule automatique

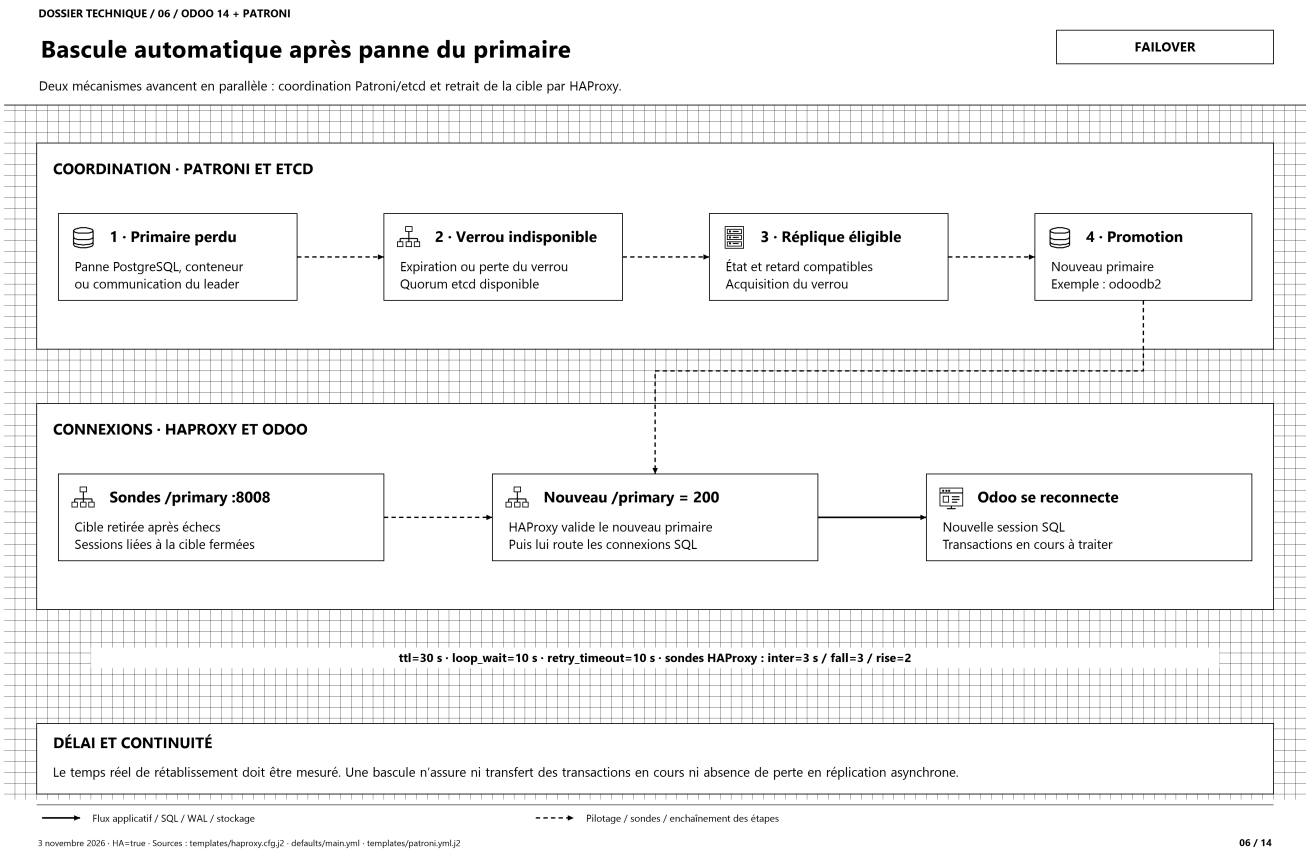


Schéma 6 – Bascule automatique après panne du primaire

Lecture du schéma 6

Deux mécanismes avancent en parallèle : la coordination Patroni/etcd et le retrait de la cible par HAProxy.

Coordination, en quatre étapes :

1. *Primaire perdu* : panne de PostgreSQL, du conteneur, ou de la communication du leader.
2. *Verrou indisponible* : le verrou expire ou se perd, alors que le quorum etcd reste disponible.
3. *Réplique éligible* : une réplique dont l'état et le retard sont compatibles acquiert le verrou.
4. *Promotion* : elle devient le nouveau primaire (exemple : odoodb2).

Connexions : les sondes `/primary` échouent et HAProxy retire la cible, en fermant les sessions qui lui étaient liées. Dès que le nouveau primaire répond 200, HAProxy lui route les connexions SQL. Odoo ouvre alors une nouvelle session ; les transactions en cours sont à traiter par l'application.
Réglages déclarés : `ttl=30 s`, `loop_wait=10 s`, `retry_timeout=10 s` ; sondes HAProxy `inter=3 s`, `fall=3`, `rise=2`. Ces valeurs ne constituent pas un temps de rétablissement mesuré. Une bascule ne transfère pas les transactions en cours et, en réplication asynchrone, ne garantit pas l'absence de perte.

3.5 Quorum et partitions réseau

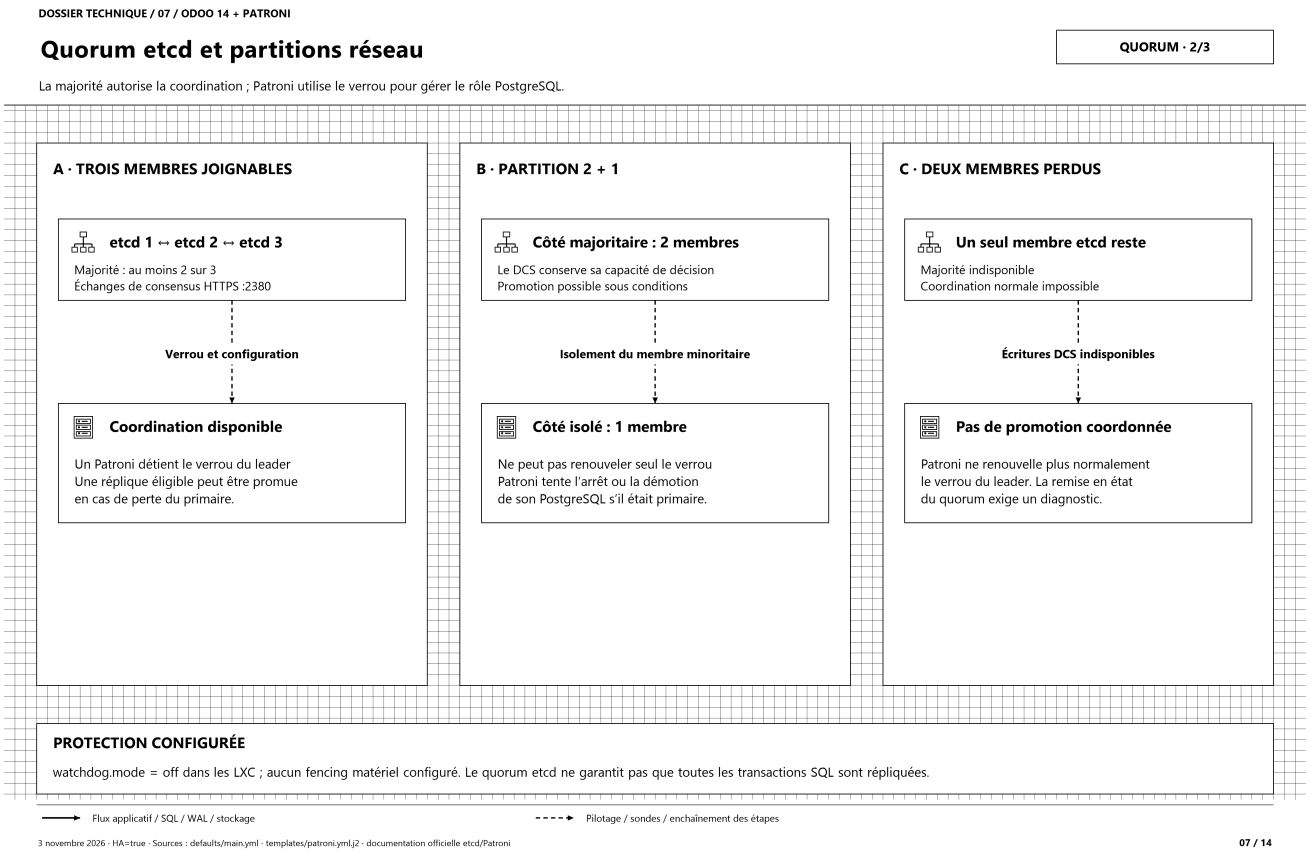


Schéma 7 – Quorum etcd et partitions réseau

Lecture du schéma 7

La majorité autorise la coordination ; Patroni utilise le verrou pour gérer le rôle de PostgreSQL.

- **A – Trois membres joignables** : la majorité (au moins 2 sur 3) est atteinte, par des échanges de consensus en HTTPS sur 2380. Un Patroni détient le verrou du leader, et une réplique éligible peut être promue si le primaire est perdu.
- **B – Partition 2 + 1** : le côté majoritaire garde sa capacité de décision ; une promotion y reste possible sous conditions. Le membre isolé ne peut pas renouveler seul le verrou. S'il portait le primaire, Patroni tente d'arrêter ou de rétrograder son PostgreSQL.
- **C – Deux membres perdus** : la majorité est indisponible et la coordination normale impossible. Aucune promotion coordonnée n'a lieu, et le dernier serveur passe en lecture seule. Rétablir le quorum exige un diagnostic.
- **Protection** : `watchdog.mode = off` dans les LXC ; aucun fencing matériel n'est configuré. Le quorum etcd ne garantit pas que toutes les transactions SQL sont répliquées.

3.6 Bascule planifiée

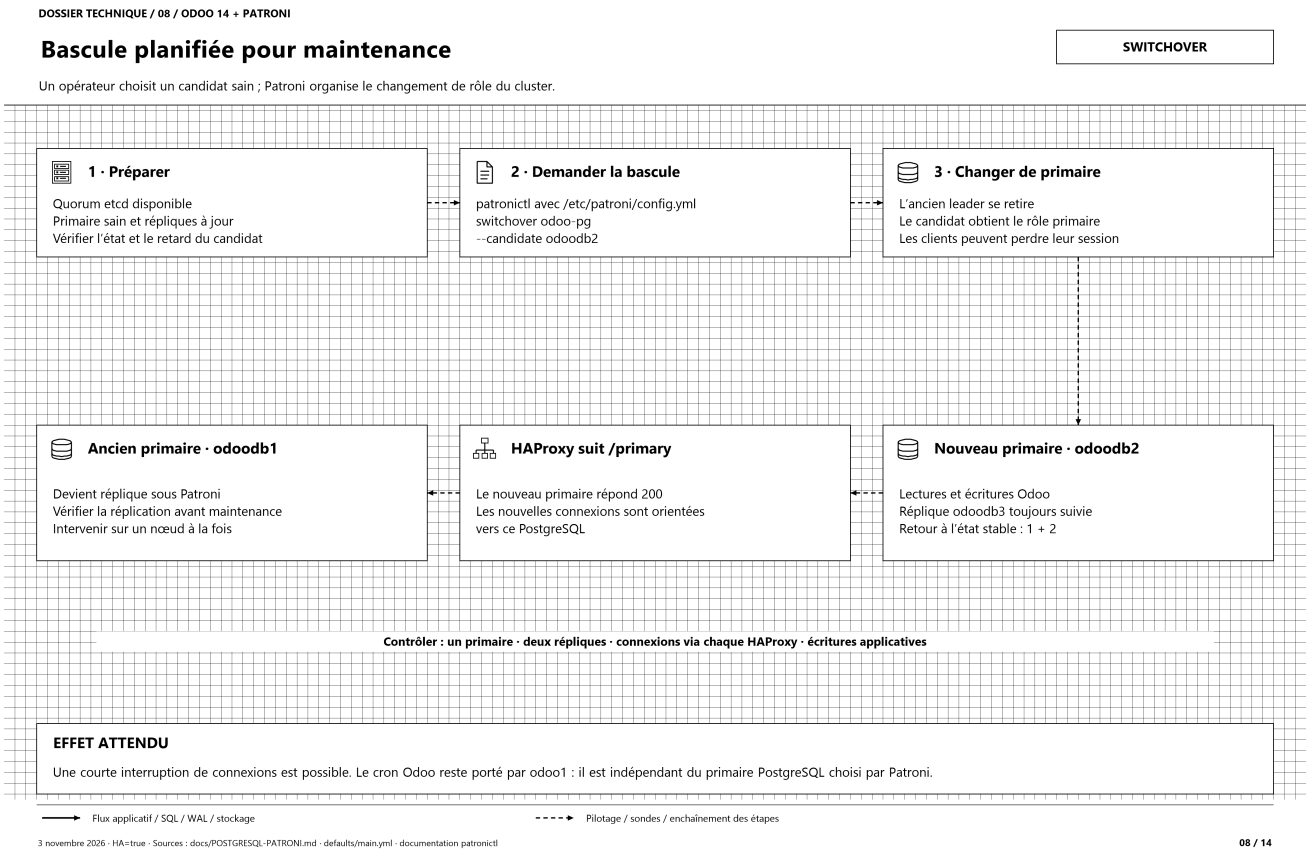


Schéma 8 – Bascule planifiée pour maintenance

Lecture du schéma 8

Un opérateur choisit un candidat sain ; Patroni organise le changement de rôle du cluster.

1. *Préparer* : vérifier que le quorum etcd est disponible, que le primaire est sain et que les répliques sont à jour, puis contrôler l'état et le retard du candidat.
2. *Demander la bascule* :

```
patronictl -c /etc/patroni/config.yml switchover odoo-pg --candidate odoodb2
```

3. *Changer de primaire* : l'ancien leader se retire et le candidat obtient le rôle de primaire. Les clients peuvent perdre leur session.

Résultat : l'ancien primaire odoodb1 devient réplique ; on vérifie la réplication avant toute maintenance, et on intervient sur un nœud à la fois. HAProxy suit `/primary` : le nouveau primaire odoodb2 répond 200 et reçoit les nouvelles connexions ; odoodb3 le suit. On revient à l'état stable d'un primaire et deux répliques.
À contrôler : un primaire, deux répliques, des connexions via chaque HAProxy et des écritures applicatives. Une courte interruption des connexions est possible. Le cron d'Odoo reste porté par odoo1, indépendamment du primaire choisi par Patroni.

4 Réseau et sécurité des flux

Chaque conteneur est relié au LAN en **IPVLAN L2**, sur la carte de son hôte. Le réseau des hôtes rejette les adresses MAC inconnues : macvlan et veth sur bridge n'obtenaient aucune réponse de la passerelle, alors qu'IPVLAN réutilise la MAC de l'hôte. L'hôte garde sa carte, qui porte sa route par défaut. En contrepartie, **un hôte ne joint pas son propre conteneur** : les contrôles se font depuis le poste de contrôle, ou par `lxc-attach`.

Port	Flux
22/TCP	Poste de contrôle vers les hôtes et les conteneurs
8069/TCP	Utilisateurs vers chaque Odoo (interface web)
8072/TCP	Utilisateurs vers chaque Odoo (bus de notifications)
5000/TCP	Odoo et sauvegarde vers les trois HAProxy
5432/TCP	HAProxy vers PostgreSQL ; réplication et `pg_rewind` entre serveurs
8008/HTTP	HAProxy vers l'API Patroni ; bascules
2379, 2380/HTTPS	etcd : clients (Patroni) et échanges entre membres
7000/HTTP	Page d'état HAProxy, sur `127.0.0.1` seulement
24007, 49152+/TCP	Conteneurs Odoo et hôte de sauvegarde vers les serveurs GlusterFS

Tableau 3 – Ports et flux

Aucun répartiteur HTTP n'est déployé : chaque utilisateur se connecte à l'adresse d'une instance. Une configuration nginx est fournie à part (`nginx/odoo-repartition.conf`), mais aucun playbook ne l'installe.

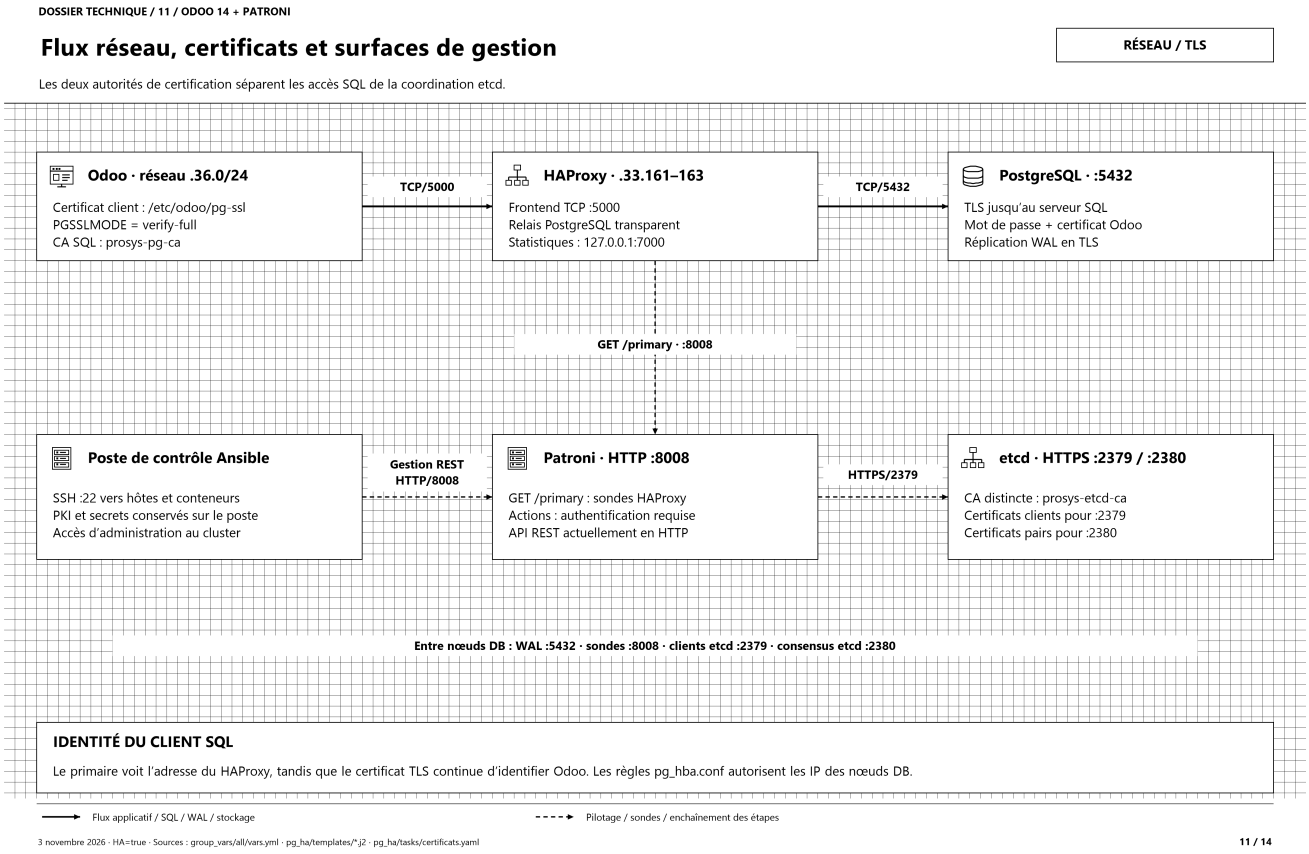


Schéma 9 – Flux réseau, certificats et surfaces de gestion

Lecture du schéma 9

Deux autorités de certification séparent les accès SQL de la coordination etcd.

- **Odoo** (réseau .36.0/24) : certificat client dans `/etc/odoo/pg-ssl`, `PGSSLMODE = verify-full`, autorité SQL `prosys-pg-ca`.
- **HAProxy** (.33.161 à .163) : frontal TCP sur 5000, relais PostgreSQL transparent, statistiques sur `127.0.0.1:7000`.
- **PostgreSQL** (5432) : TLS jusqu'au serveur SQL, mot de passe et certificat d'Odoo exigés, réplication des WAL en TLS.
- **Patroni** (HTTP 8008) : `GET /primary` pour les sondes de HAProxy ; les actions exigent une authentification ; l'API reste en HTTP, non chiffrée.
- **etcd** (HTTPS 2379 et 2380) : autorité distincte `prosys-etcd-ca`, certificats clients pour 2379, certificats de pairs pour 2380. Avec une autorité commune, le certificat client d'un conteneur Odoo ouvrirait etcd, donc la configuration de PostgreSQL.
- **Poste de contrôle Ansible** : SSH sur 22 vers les hôtes et les conteneurs ; il conserve la PKI et les secrets.
- **Identité du client SQL** : le primaire voit l'adresse du HAProxy, mais le certificat TLS continue d'identifier Odoo. Les règles `pg_hba.conf` autorisent les adresses des nœuds de données.

5 Installation et déploiement d'Odoo

5.1 Prérequis du poste de contrôle

- **Ansible 2.10.x** et les collections épinglées dans `requirements.yml` (`community.general 1.3.6`, `community.postgresql 1.1.1`, `ansible.posix 1.1.1`). `ansible.posix 1.1.1` refuse de se charger avec un Ansible plus récent.
- **Accès SSH par clé** aux six hôtes ; le rôle `lxc_lan` injecte `~/.ssh/id_rsa.pub` dans les conteneurs.
- **Secrets** chiffrés dans `group_vars/all/vault.yml`.
- **Autorités de certification** dans `pki/`, sur le poste de contrôle et hors du dépôt. Le rôle `pg_ha` refuse d'en créer une nouvelle : il faut lancer les playbooks depuis le poste qui la détient.
- **Hors du dépôt** : le volume `sre` accepte les adresses des conteneurs Odoo, et les ports du tableau des flux sont ouverts.

Secret	Usage
`vault_odoo_admin_password`	Mot de passe maître d'Odoo (gestion des bases)
`vault_odoo_db_password`	Rôle PostgreSQL `odoo`, partagé par la base et les Odoo
`vault_pg_replication_password`	Rôle `replicator` (flux WAL)
`vault_pg_patroni_superuser_password`	Superutilisateur `postgres`, pour `pg_rewind`
`vault_pg_patroni_rest_password`	Compte `patroni` de l'API REST

Tableau 4 – Secrets du vault

5.2 Ordre de déploiement

L'ordre est contraignant :

```
ansible-playbook playbooks/ansi2-snapchot.yaml              # 1. conteneurs LXC Debian 12
```

```
ansible-playbook playbooks/reseau-lxc.yaml --check --diff     # 2. voir les changements
    reseau
ansible-playbook playbooks/reseau-lxc.yaml                    #    IPVLAN + SSH (redemarre
    les LXC)
ansible-playbook playbooks/ansi_odoo.yaml --ask-vault-pass    # 3. base, puis Odoo
```

Le rôle de chaque playbook, ses étapes et ses précautions sont détaillés à la section 6. Le schéma suivant résume le parcours d'installation d'Odoo que réalise `ansi_odoo.yaml`.

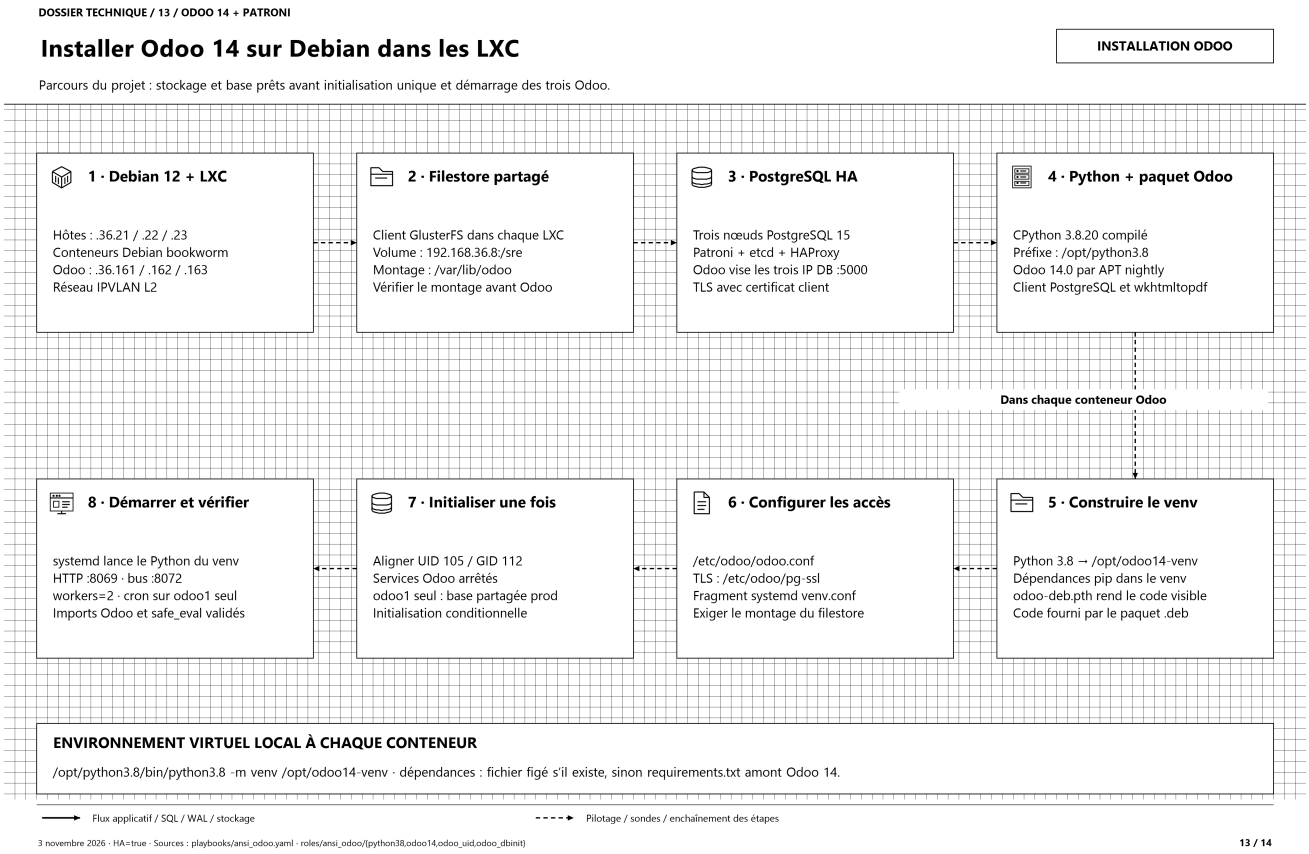


Schéma 10 – Installer Odoo 14 sur Debian dans les LXC

Lecture du schéma 10

Parcours du projet : stockage et base prêts avant une initialisation unique et le démarrage des trois Odoo.

1. *Debian 12 et LXC* : hôtes .36.21, .22 et .23 ; conteneurs Debian bookworm .36.161, .162 et .163 ; réseau IPVLAN L2.
2. *Filestore partagé* : client GlusterFS dans chaque conteneur, volume `192.168.36.8:/sre` monté sur `/var/lib/odoo`, vérifié avant Odoo.
3. *PostgreSQL HA* : trois nœuds PostgreSQL 15 avec Patroni, etcd et HAProxy ; Odoo vise les trois adresses sur 5000, en TLS avec certificat client.
4. *Python et paquet Odoo* : CPython 3.8.20 compilé dans `/opt/python3.8` ; Odoo 14.0 par le dépôt APT nightly ; client PostgreSQL et wkhtmltopdf.
5. *Construire le venv* : `/opt/odoo14-venv`, dépendances pip dans le venv ; `odoo-deb.pth` rend visible le code fourni par le paquet `.deb`.

6. *Configurer les accès* : `/etc/odoo/odoo.conf`, certificats dans `/etc/odoo/pg-ssl`, fragment systemd `venv.conf`, montage du filestore exigé.
7. *Initialiser une fois* : UID 105 et GID 112 alignés, services Odoo arrêtés ; odoo1 seul initialise la base partagée `prod`, et seulement si elle ne l'est pas déjà.
8. *Démarrer et vérifier* : systemd lance le Python du venv ; HTTP 8069 et bus 8072 ; `workers=2` ; cron sur odoo1 seul ; imports d'Odoo et `safe_eval` validés.

Le venv est local à chaque conteneur : `/opt/python3.8/bin/python3.8 -m venv /opt/odoo14-venv`. Les dépendances viennent du fichier figé s'il existe, sinon du `requirements.txt` amont d'Odoo 14.

5.3 Pile d'exécution d'Odoo

Odoo 14 ne fonctionne pas sur le Python 3.11 de Debian 12 : son `safe_eval` rejette le bytecode de Python 3.11 (opcode `RESUME`). Aucune version de Debian ne fournit Python 3.8 ; il est donc compilé depuis les sources et confiné dans `/opt`.

Couche	Choix
Hôtes et conteneurs	Debian 12 bookworm, LXC
Interpréteur d'Odoo	CPython 3.8.20 dans `/opt/python3.8`, venv `/opt/odoo14-venv`
Odoo	Paquet `.deb` nightly 14.0, lancé par le Python du venv
Base	PostgreSQL 15 de Debian, sous Patroni
Coordination	etcd, trois membres, quorum de deux
Routage SQL	Un HAProxy par conteneur PostgreSQL, port 5000
Filestore	Client GlusterFS dans chaque conteneur Odoo, sur `/var/lib/odoo`

Tableau 5 – Pile logicielle

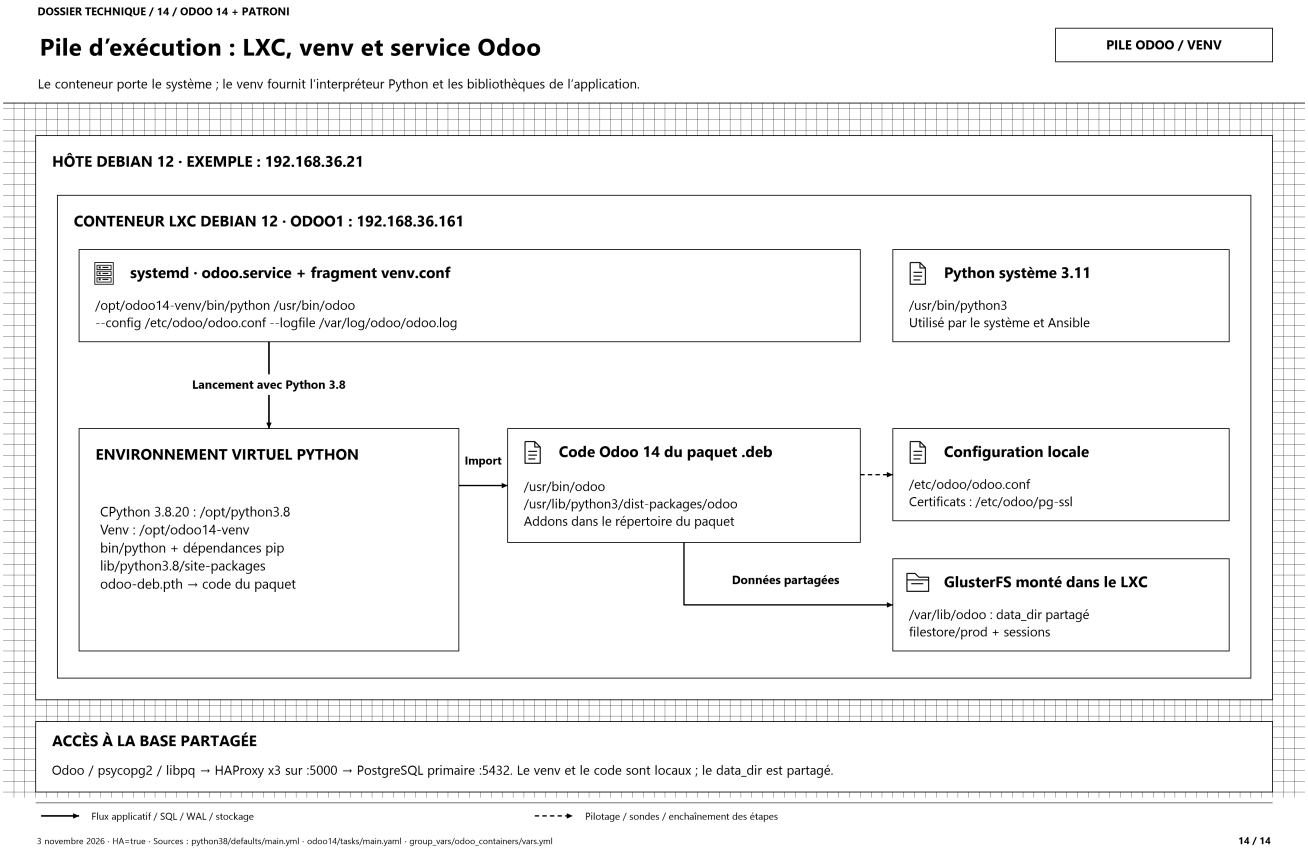


Schéma 11 – Pile d'exécution : LXC, venv et service Odoo

Lecture du schéma 11

Le conteneur porte le système ; le venv fournit l'interpréteur Python et les bibliothèques de l'application.

- Emboîtement : hôte Debian 12 (exemple 192.168.36.21), conteneur LXC odoo1 (192.168.36.161), puis environnement virtuel Python.
- **systemd** lance `odoo.service`, complété par le fragment `venv.conf` :

```
/opt/odoo14-venv/bin/python /usr/bin/odoo --config /etc/odoo/odoo.conf --logfile /
    var/log/odoo/odoo.log
```

- **Python système 3.11** (`/usr/bin/python3`) reste celui du système et d'Ansible.
- **Venv** : CPython 3.8.20 dans `/opt/python3.8`, venv `/opt/odoo14-venv`, dépendances pip dans `lib/python3.8/site-packages` ; `odoo-deb.pth` pointe vers le code du paquet.
- **Code d'Odoo 14** : fourni par le paquet `.deb` (`/usr/bin/odoo`, `/usr/lib/python3/dist-packages/odoo`, et les addons du paquet).
- **Configuration locale** : `/etc/odoo/odoo.conf` et certificats dans `/etc/odoo/pg-ssl`.
- **Données partagées** : GlusterFS monté dans le conteneur sur `/var/lib/odoo` (`data_dir`), avec `filestore/prod` et `sessions`.
- **Accès à la base** : Odoo, psycopg2 et libpq vers les trois HAProxy sur 5000, puis vers le primaire sur 5432. Le venv et le code sont locaux ; le `data_dir` est partagé.

6 Les playbooks en détail

Chaque playbook ne fait qu'orchestrer : la logique vit dans des rôles, rangés par playbook sous `roles/<playbook>/<rôle>`. Le `roles_path` de `ansible.cfg` les retrouve tous par leur seul nom. Les playbooks se lancent depuis la racine du dépôt, avec l'inventaire unique `inventory.yaml`.

Playbook	Cibles	Rôle et moment d'utilisation
`ansi2-snapchot.yaml`	les six hôtes	Créer les conteneurs LXC ; une fois, avant tout le reste
`reseau-lxc.yaml`	les six hôtes	Réseau IPVLAN et SSH des conteneurs ; après la création, ou pour réparer le réseau
`ansi_odoo.yaml`	hôtes et conteneurs	Déploiement complet ; rejouable pour remettre en conformité
`filestore-conteneur.yaml`	conteneurs et hôtes Odoo	Montage GlusterFS par les conteneurs ; joué aussi par `ansi_odoo.yaml` et `ansi_syncro.yaml`
`ansi_postgresql_ha.yaml`	les trois serveurs PostgreSQL	Reprise du cluster par Patroni, puis remise en conformité
`ansi_syncro.yaml`	hôtes, conteneurs et base	Migration des pièces jointes entre la base et le filestore
`nettoyage_odoo.yaml`	conteneurs Odoo	Réparation APT après un lancement interrompu
`nettoyage.yaml`	les six hôtes	Destruction des conteneurs et désinstallation de LXC

Tableau 6 – Vue d'ensemble des playbooks

Le playbook `ansi_sauvegarde.yaml`, qui déployait la sauvegarde, n'est plus présent dans le dépôt (section **??**).

6.1 `ansi2-snapchot.yaml` – créer les conteneurs

Objectif. Installer LXC sur chaque hôte et y créer le conteneur Debian 12 déclaré dans `host_vars/` : `devodoo14` sur les hôtes Odoo, `devpg15` sur les hôtes PostgreSQL. La liste des conteneurs vient de `common/group_vars/local_dev.yml`, qui la dérive de `lxc_container_name` hôte par hôte.

Déroulement (rôle `install`, sur chaque hôte).

1. Détecter le système et installer les paquets LXC. Une branche Rocky/RHEL (EPEL, SELinux permissif) est conservée pour l'ancien lab.
2. Démarrer le réseau LXC (`lxcbr0`) et le service LXC, puis détruire les conteneurs de diagnostic laissés par un essai précédent.
3. Mesurer l'espace libre sous `/var/lib/lxc`. Si la partition est trop petite, déporter le stockage LXC sous `/srv` : copie du contenu, montage et inscription dans `/etc/fstab`. Le rôle exige ensuite assez d'espace pour le système du conteneur.
4. Supprimer les conteneurs obsolètes listés dans `stale_containers`, si cette variable est définie.
5. Créer et démarrer le conteneur Debian 12 *bookworm* depuis le modèle `download`. En cas d'échec, le rôle rejoue `lxc-create` sans `--quiet` et affiche l'état du disque, du réseau et le journal de création.
6. Activer le démarrage automatique du conteneur.

Lancement.

```
ansible-playbook playbooks/ansi2-snapchot.yaml
```

Précautions. Le conteneur créé n'a pas encore son réseau définitif : il faut enchaîner avec `reseau-lxc.yaml`. Renseigner `stale_containers` détruit des conteneurs : sauvegarder avant.

6.2 `reseau-lxc.yaml` – relier les conteneurs au réseau

Objectif. Exposer chaque conteneur directement sur le LAN en IPVLAN L2, puis amorcer son accès SSH pour qu'Ansible puisse le gérer. C'est le seul playbook qui joue le rôle `lxc_lan` : `ansi_odoo.yaml` suppose qu'il a déjà été passé.

Déroulement (rôle `lxc_lan`, sur chaque hôte).

1. Contrôler la description : mode réseau et adressage présents dans `host_vars/`, passerelle dans le sous-réseau du conteneur, carte réseau présente. Le rôle refuse de céder au conteneur la carte qui porte la route par défaut de l'hôte.
2. Sauvegarder, une seule fois, la configuration LXC d'origine, puis retirer les réglages des anciens modes (macvlan, adresse MAC figée).
3. Écrire l'interface du conteneur en `ipvlan` sur la carte de l'hôte, et son adresse IP statique dans le conteneur (`systemd-networkd`, sans DHCP). La sortie NAT de secours par `lxcbr0` est retirée, car `lxc_egress_nat=false`.
4. Arrêter puis redémarrer le conteneur, attendre l'état `RUNNING` et la configuration d'`eth0`. En cas d'échec, le rôle affiche la configuration et le journal LXC.
5. Installer `openssh-server`, `python3` et `python3-apt`, déposer la clé publique du poste de contrôle, puis attendre que le port 22 du conteneur réponde, vu du poste de contrôle.

Lancement.

```
ansible-playbook playbooks/reseau-lxc.yaml --check --diff        # voir ce qui changerait
ansible-playbook playbooks/reseau-lxc.yaml
ansible-playbook playbooks/reseau-lxc.yaml --limit 192.168.36.21  # un seul hote
```

Précautions. Le playbook **redémarre les conteneurs**. En IPVLAN, un hôte ne joint pas son propre conteneur : les tests se font depuis le poste de contrôle. Le poste doit avoir la version à jour du rôle : une version ancienne réécrirait le mode macvlan et casserait un conteneur déjà passé en IPVLAN. La procédure manuelle équivalente est décrite dans `docs/BOOTSTRAP-CONTENEUR-MANUEL.md`.

6.3 `ansi_odoo.yaml` – déploiement complet

Objectif. Déployer la base, puis les trois instances Odoo, dans un ordre contraignant. Rejoué sur une installation existante, il remet chaque élément en conformité.

Play	Cible	Effet
0/5	poste de contrôle	Contrôle d'environnement : Ansible entre 2.10 et 2.11, collections de `requirements.yml` présentes. Tag `never` : à demander par `--tags all,preflight`
1/5	les six hôtes	`chrony` remet l'horloge à l'heure : une horloge fausse fait échouer `apt` dans les conteneurs
1bis/5	conteneurs, puis hôtes Odoo	Import de `filestore-conteneur.yaml` : le volume doit être monté avant l'installation d'Odoo
2/5, 2bis/5	`db_containers`, puis `db_replicas`	`pg_server` : primaire, puis réserves en streaming. Plays sautées quand `pg_ha_enabled: true`
2ter/5	`db_cluster`	`pg_ha` : Patroni, etcd et HAProxy sur les trois serveurs ensemble ; un échec arrête les trois. Sur un cluster déjà repris, ne fait que vérifier
3/5	`odoo_containers`	`odoo14`, puis `odoo_uid` : Odoo installé et configuré, service laissé arrêté
4/5	`odoo_primary`	`odoo_dbinit` : `odoo -i base --without-demo=all` sur la base partagée, par odoo1 seul, avec son certificat client
5/5	`odoo_containers`	Démarrage et activation d'Odoo, attente du port 8069 (120 s au plus)

Tableau 7 – Plays du playbook `ansi_odoo.yaml`

Déroulement. Le play 3/5 fait l'essentiel du travail dans chaque conteneur Odoo :

- sources APT cohérentes (rôle `base_debian`) et CPython 3.8.20 compilé dans `/opt/python3.8` (rôle `python38`) ;
- dépôt APT nightly d'Odoo 14 et installation du paquet `.deb` ;
- environnement virtuel `/opt/odoo14-venv` sur Python 3.8. Les dépendances viennent d'un fichier de versions figées s'il existe sur le poste de contrôle, sinon du `requirements.txt` amont ;
- `odoo-deb.pth` rend le code du paquet visible depuis le venv ; une enveloppe `odoo` et un fragment systemd font lancer le service par le Python du venv ;
- refus de configurer Odoo si `/var/lib/odoo` n'est pas le volume GlusterFS, et exigence de ce montage au démarrage ;
- vérification qu'Odoo s'importe sous Python 3.8 et que `safe_eval` accepte ses expressions ;
- dépôt du certificat client du nœud dans `/etc/odoo/pg-ssl`, puis de `/etc/odoo/odoo.conf` ;
- contrôle que la base partagée écoute et que ce nœud est autorisé à s'y connecter ; Odoo reste arrêté jusqu'à l'initialisation.

Le rôle `odoo_uid` aligne ensuite l'UID et le GID du compte `odoo` (105/112) sur les trois conteneurs. Trois `odoo -i base` concurrents sur la même base corrompraient le registre : c'est pour cela que l'initialisation (4/5) est confiée à odoo1 seul et précède le démarrage commun (5/5).

Lancement.

```
ansible-playbook playbooks/ansi_odoo.yaml --ask-vault-pass
ansible-playbook playbooks/ansi_odoo.yaml --ask-vault-pass --tags all,preflight  # avec le
    controle du poste
ansible-playbook playbooks/ansi_odoo.yaml --ask-vault-pass --tags horloge        # horloge
    seule
ansible-playbook playbooks/ansi_odoo.yaml --ask-vault-pass --tags filestore      # montage
    seul
```

Précautions. Les conteneurs doivent exister et avoir leur réseau (`ansi2-snapchot.yaml`, puis `reseau-lxc.yaml`). Le vault doit être renseigné, et le volume GlusterFS doit accepter les adresses

des conteneurs Odoo. Le commentaire d'en-tête du playbook cite encore les adresses de l'ancien lab (192.168.96.x) : c'est l'inventaire qui fait foi.

6.4 `ansi_postgresql_ha.yaml` – haute disponibilité de la base

Objectif. Installer etcd, Patroni et HAProxy dans les trois conteneurs PostgreSQL. Au premier passage, il reprend le cluster installé par `pg_server`, sans perte ni recopie. Aux passages suivants, il remet en conformité, sans coupure.

Déroulement (rôle `pg_ha`, sur les trois serveurs ensemble).

1. *Contrôles* : trois serveurs, secrets présents, autorité de certification existante. Le rôle refuse d'en créer une nouvelle, car Odoo rejetterait alors tous les serveurs.
2. *Paquets* : `patroni`, `etcd-server` et `haproxy`, installés sans démarrer leurs services.
3. *Constat et décision* : l'état de chaque serveur est lu sans rien arrêter (tableau 8).
4. *Certificats et etcd* : certificats PostgreSQL et autorité etcd distincte ; etcd démarré sur les trois serveurs en même temps, puis vérifié sain avec ses trois membres.
5. *Patroni et reprise* : configuration de Patroni, puis reprise du primaire, confirmée par etcd, puis des réserves. Le PostgreSQL de Debian est retiré : unité masquée, `start.conf` désactivé, anciens fichiers de `pg_server` renommés en `.avant-patroni`.
6. *Relance* de Patroni, en pause et un serveur à la fois, seulement s'il tourne avec une ancienne liste etcd.
7. *Comptes et configuration* : mot de passe de `postgres` (pour `pg_rewind`), rôles `replicator` et `odoo`, base `prod` ; configuration dynamique d'etcd remise en conformité avec `pg_ha_dcs`, en conservant une pause posée à la main.
8. *HAProxy, puis vérification* : chaque Patroni joint les trois membres etcd ; un primaire et deux réserves avec moins de 16 Mo de retard ; une vraie connexion TLS à travers chaque HAProxy aboutit au primaire.

Décision	Quand	Effet
`demarrer`	serveur déjà repris par Patroni	Patroni est démarré s'il ne tourne pas
`primaire`	premier passage, odoodb1, qui n'est pas en réserve	Debian arrête PostgreSQL ; Patroni le redémarre et adopte ses données
`reserve`	même cluster que le primaire, en réserve	repris en réserve, sans recopie
`recopie`	cluster étranger sans aucune base	vidé puis recopié depuis le primaire
`refus`	tout autre cas	arrêt du playbook, avant toute modification

Tableau 8 – Décision de reprise, par serveur

Lancement.

```
ansible-playbook playbooks/ansi_postgresql_ha.yaml --ask-vault-pass
```

Précautions. Jamais de `--limit` : etcd et la reprise exigent les trois serveurs ensemble. Lancer depuis le poste qui détient `pki/` et le vault, avec les deux secrets Patroni. Ouvrir 2379, 2380, 8008 et 5432 entre les serveurs, et 5000 depuis le réseau Odoo. Au premier passage, Odoo perd la base quelques secondes. Il faut ensuite passer sans attendre `pg_ha_enabled: true`, puis rejouer `ansi_odoo.yaml` : sinon, après une bascule, Odoo resterait sur un serveur en lecture seule.

6.5 `nettoyage_odoo.yaml` – réparer un conteneur Odoo

Objectif. Remettre à plat les conteneurs Odoo après un lancement interrompu, sans les recréer.

Déroulement par défaut, non destructif (rôle `odoo_cleanup`) : arrêt d'Odoo ; libération des verrous `dpkg`/`apt` résiduels ; reprise d'une transaction `dpkg` interrompue ; réparation des paquets laissés incohérents ; réalignement des versions par l'outil `apt-realign` ; réparation des dépendances ; vidage du cache ; résumé de l'état du conteneur.

Option `full_reset`. Avec `-e full_reset=true`, le rôle purge en plus Odoo et PostgreSQL, puis supprime leurs répertoires (`/var/lib/odoo`, `/etc/odoo`, `/var/log/odoo`, `/var/lib/postgresql`, `/etc/postgresql`) et le dépôt nightly.

Risque : `full_reset` et filestore partagé

Dans l'architecture actuelle, `/var/lib/odoo` est le point de montage du volume GlusterFS commun. Supprimer ce répertoire effacerait le contenu du filestore partagé par les trois instances. Le garde-fou `destroy_shared_db` ne protège que les hôtes du groupe `db_containers`, que ce playbook ne vise pas. Ne pas utiliser `full_reset` en l'état sans avoir d'abord démonté le volume et vérifié une sauvegarde.

Lancement.

```
ansible-playbook playbooks/nettoyage_odoo.yaml --ask-vault-pass                    #
    reparation APT
ansible-playbook playbooks/nettoyage_odoo.yaml --ask-vault-pass -e full_reset=true  #
    DESTRUCTIF
```

6.6 `nettoyage.yaml` – tout désinstaller

Objectif. Repartir de zéro au niveau des hôtes.

Déroulement (rôle `cleanup`, sur les six hôtes) : destruction des conteneurs LXC ; arrêt et désactivation des services LXC ; suppression des paquets, des répertoires et des fichiers créés pour LXC ; en option (`-e remove_epel=true`), suppression du dépôt EPEL ; sur Rocky/RHEL, retour de SELinux en mode `enforcing`.

Lancement.

```
ansible-playbook playbooks/nettoyage.yaml
```

Précautions. Le groupe visé comprend aussi les trois hôtes PostgreSQL : le playbook détruit les conteneurs `devpg15`, donc les données de la base. Sauvegarder d'abord. Pour repartir de zéro, enchaîner `nettoyage.yaml`, `ansi2-snapchot.yaml`, `reseau-lxc.yaml`, puis `ansi_odoo.yaml` et la séquence de passage à Patroni.

7 Vérification et exploitation

7.1 Vérification après déploiement

Depuis le poste de contrôle, puisqu'un hôte ne joint pas son propre conteneur :

```
# Odoo actif sur les trois conteneurs
ansible odoo_containers -m shell -a 'systemctl is-active odoo'

# /var/lib/odoo est le volume GlusterFS (attendu : fuse.glusterfs 192.168.36.8:/sre)
ansible odoo_containers -m shell -a 'findmnt -n -o FSTYPE,SOURCE /var/lib/odoo'

# un Leader et deux Replica en streaming, retard proche de 0
ansible odoodb1 -m shell -a 'patronictl -c /etc/patroni/config.yml list'

# interface web des trois instances (attendu : 200)
for i in 161 162 163; do curl -sI -o /dev/null -w "$i -> %{http_code}\n" \
  http://192.168.36.$i:8069/web/login; done
```

Le compte créé par `odoo -i base` est `admin` / `admin` ; son mot de passe est à changer dès la première connexion. Le mot de passe du vault `vault_odoo_admin_password` est le mot de passe maître de gestion des bases, et le gestionnaire de bases web est désactivé (`list_db = false`).

7.2 Exploitation courante

Sur un conteneur PostgreSQL :

```
patronictl -c /etc/patroni/config.yml list                    # etat du cluster
patronictl -c /etc/patroni/config.yml switchover odoo-pg      # bascule planifiee
patronictl -c /etc/patroni/config.yml restart odoo-pg         # redemarrer PostgreSQL
patronictl -c /etc/patroni/config.yml reinit odoo-pg odoodb3  # recopier une reserve
journalctl -u patroni -f                                      # journal de Patroni
ssh -L 7000:127.0.0.1:7000 root@192.168.33.161                # page d'etat HAProxy
```

— C'est Patroni qui démarre et arrête PostgreSQL : on n'utilise plus `systemctl restart postgresql`, `pg_ctlcluster` ni `pg_ctl`, et l'unité Debian est masquée.

— Un paramètre commun se change dans `pg_ha_dcs`, puis on rejoue le playbook. Un `patronictl edit-config` serait écrasé au passage suivant.

— `/etc/patroni/config.yml` ne se corrige pas à la main : on rejoue le playbook, qui relance Patroni en pause si la liste etcd a changé.

8 Évolution de l'architecture

Les schémas 15 à 24 ont été produits avant l'édition du 3 novembre 2026. Ils décrivent des états antérieurs du dépôt : haute disponibilité non activée, GlusterFS monté sur l'hôte puis passé au conteneur par un bind. Ils sont conservés parce qu'ils expliquent les choix actuels. Chaque lecture précise ce qui a changé depuis ; en cas de divergence, les schémas 1 à 14 font foi.

8.1 Réseau des conteneurs LXC

Ces trois vues décrivent le raccordement IPVLAN, toujours en vigueur, et les chemins réseau d'un utilisateur et d'Odoo.

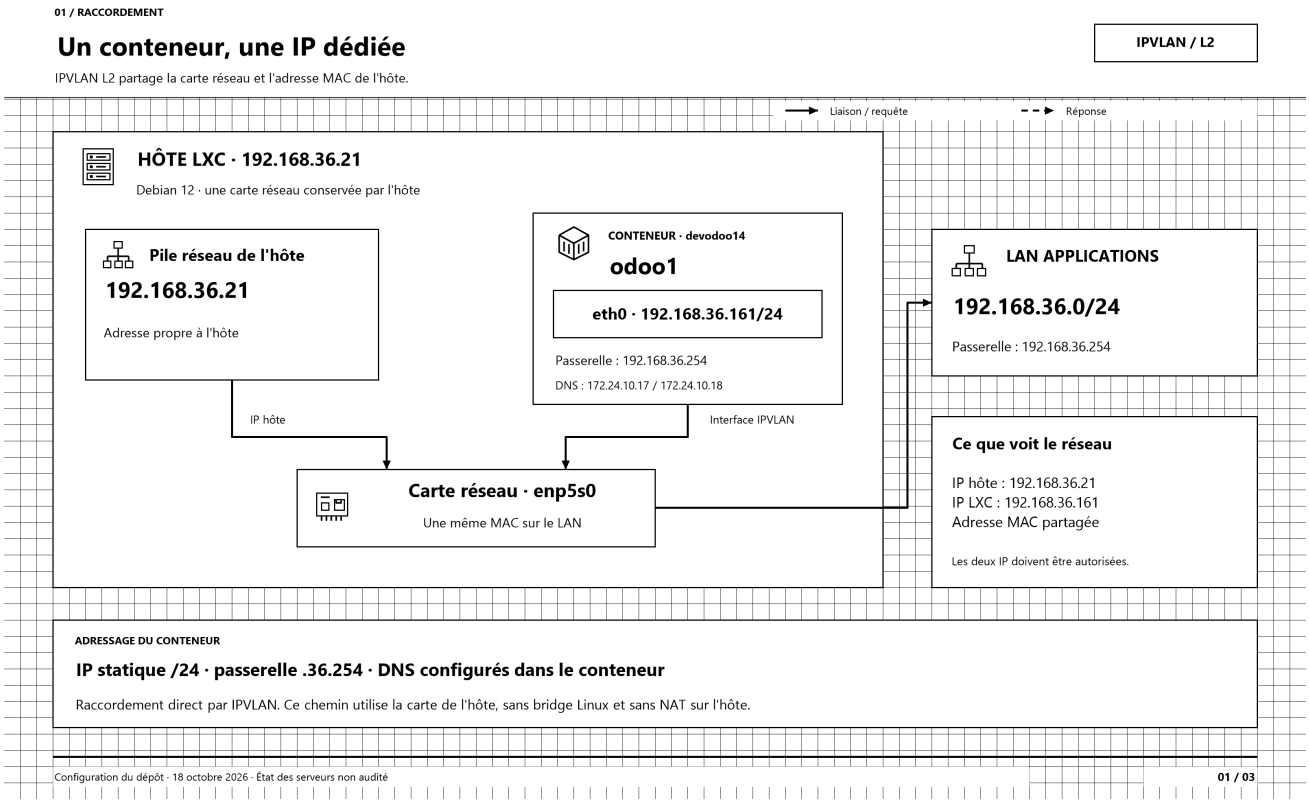


Schéma 12 – Raccordement IPVLAN : un conteneur, une IP dédiée

Lecture du schéma 12

IPVLAN L2 partage la carte réseau et l'adresse MAC de l'hôte.

- L'hôte LXC 192.168.36.21 (Debian 12) garde sa carte `enp5s0` et son adresse propre, 192.168.36.21.
- Le conteneur `devodoo14` (odoo1) reçoit une adresse statique : `eth0` en 192.168.36.161/24, passerelle 192.168.36.254, DNS 172.24.10.17 et 172.24.10.18, configurés dans le conteneur.
- L'adresse de l'hôte et l'interface IPVLAN passent par la même carte. Sur le LAN applications (192.168.36.0/24), le réseau voit donc deux adresses IP derrière une seule adresse MAC : les deux adresses doivent y être autorisées.
- Le raccordement est direct : ni bridge Linux, ni NAT sur l'hôte.

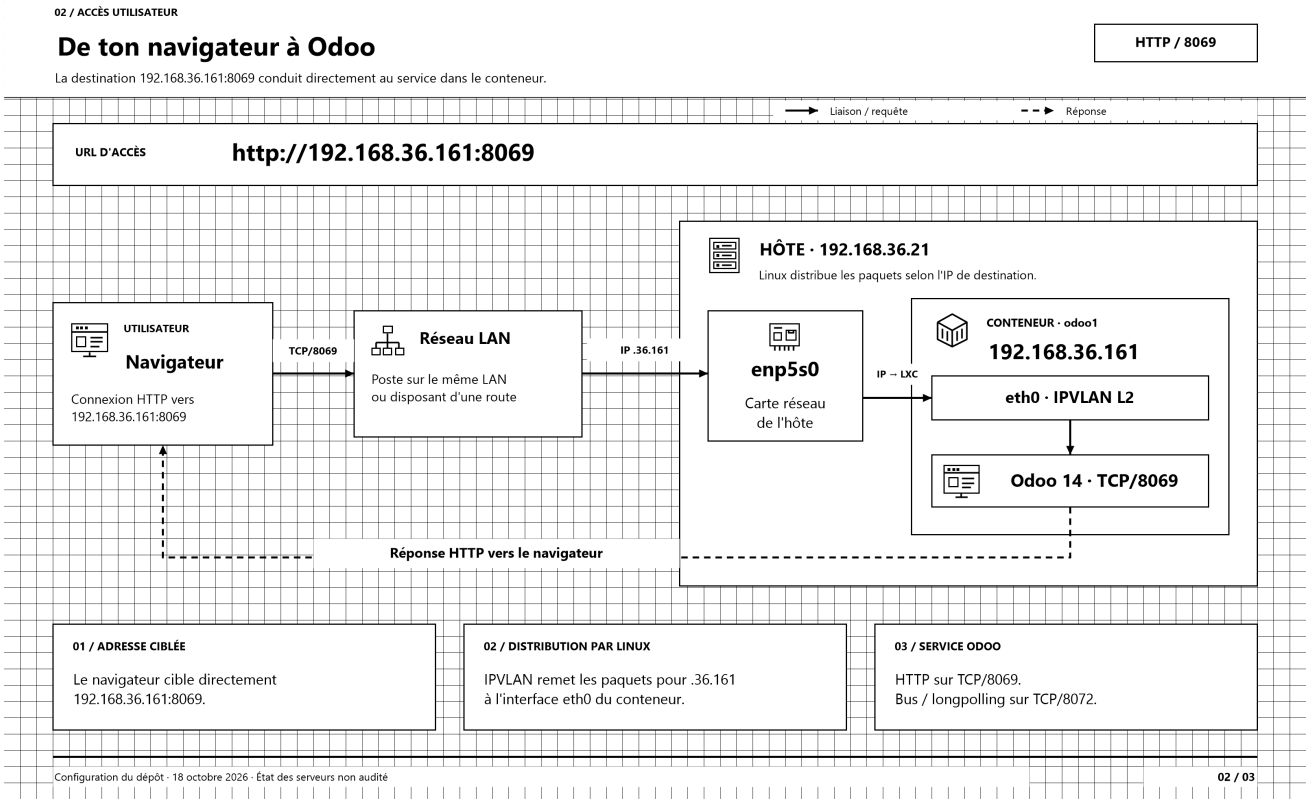


Schéma 13 – Accès utilisateur : du navigateur à Odoo

Lecture du schéma 13

La destination 192.168.36.161 :8069 conduit directement au service dans le conteneur.

1. *Adresse ciblée* : depuis un poste du même LAN, ou disposant d'une route, le navigateur ouvre `http://192.168.36.161:8069`.
2. *Distribution par Linux* : le paquet arrive sur la carte `enp5s0` de l'hôte 192.168.36.21. Linux le distribue selon son adresse de destination : IPVLAN remet ce qui est destiné à .36.161 à l'interface `eth0` du conteneur.
3. *Service Odoo* : Odoo 14 répond en HTTP sur TCP/8069 ; le bus de notifications (longpolling) utilise TCP/8072. La réponse HTTP repart vers le navigateur.

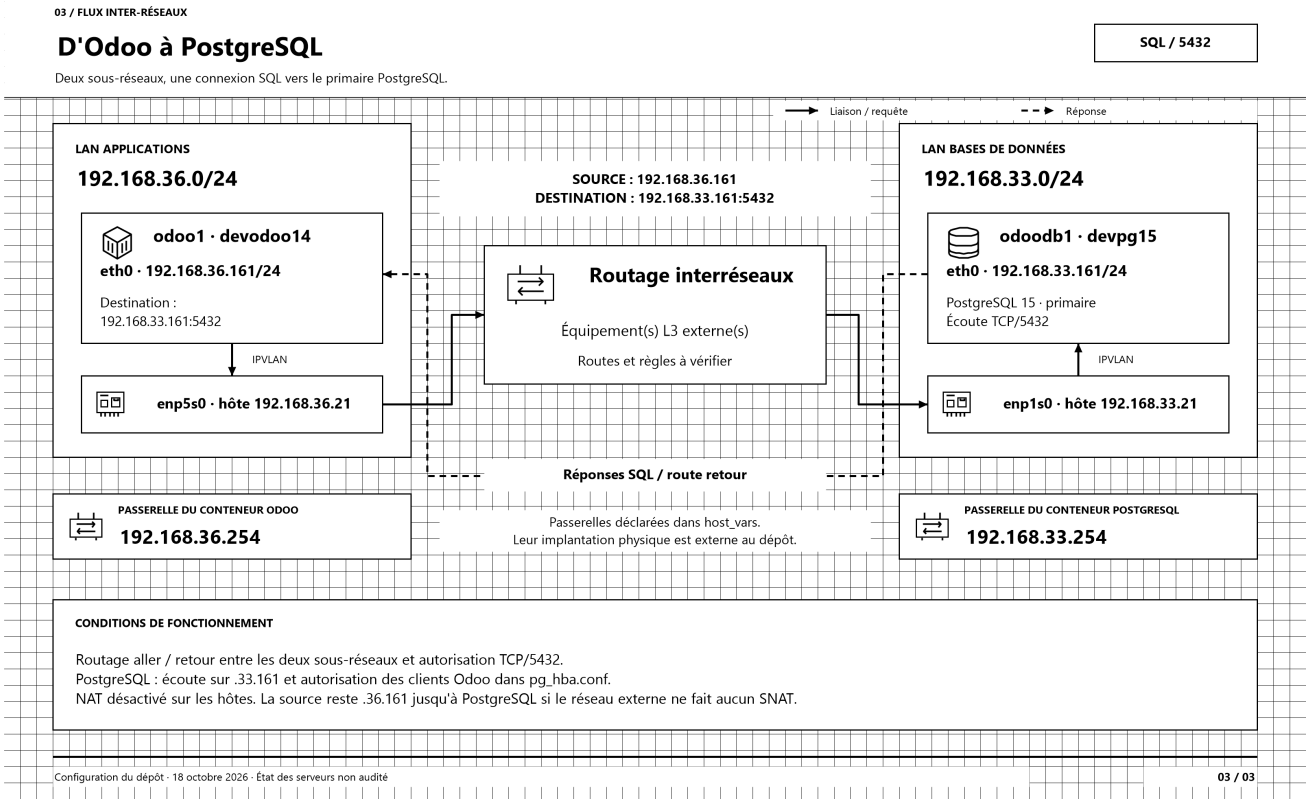


Schéma 14 – Flux Odoo vers PostgreSQL : deux sous-réseaux

Lecture du schéma 14

Deux sous-réseaux, une connexion SQL vers le primaire PostgreSQL.

- Côté applications : odoo1 (`devodoo14`, `eth0` en 192.168.36.161/24, hôte .36.21 par `enp5s0`) vise 192.168.33.161 :5432.
- Côté bases de données : odoodb1 (`devpg15`, `eth0` en 192.168.33.161/24, hôte .33.21 par `enp1s0`) porte le PostgreSQL 15 primaire, à l'écoute sur TCP/5432.
- Entre les deux, un ou plusieurs équipements L3 externes assurent le routage. Les passerelles .36.254 et .33.254 sont déclarées dans `host_vars/` ; leur implantation physique est hors du dépôt.
- Conditions de fonctionnement : routage aller et retour, autorisation de TCP/5432, écoute de PostgreSQL sur .33.161 et autorisation des clients Odoo dans `pg_hba.conf`. Le NAT est désactivé sur les hôtes.

8.2 Filestore GlusterFS

Ces trois vues décrivent le volume partagé et l'ancien modèle de montage, dans lequel l'hôte montait le volume puis le passait au conteneur.

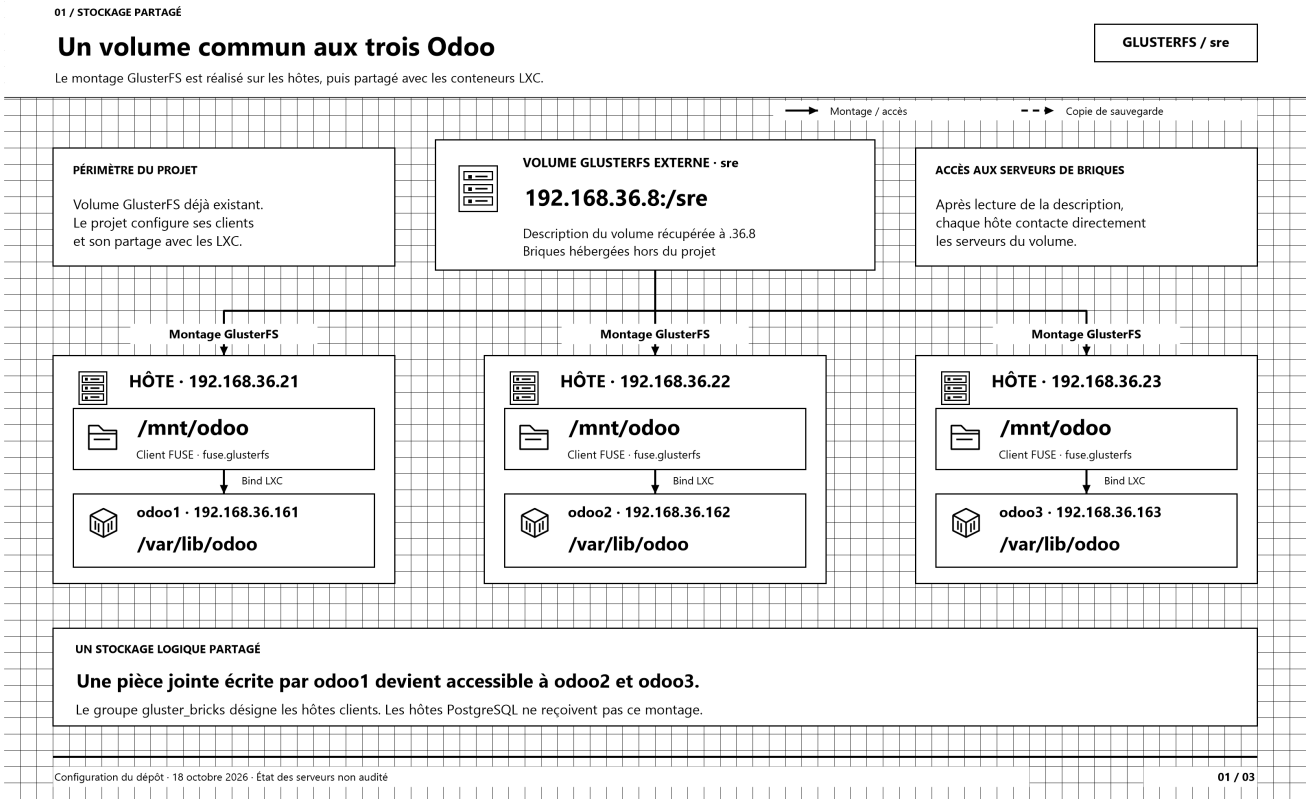


Schéma 15 – Volume GlusterFS partagé entre les trois Odoo

Lecture du schéma 15

Un stockage logique unique pour les pièces jointes des trois instances.

- **Périmètre** : le volume GlusterFS existe déjà, hors du projet. Le projet configure ses clients et son partage avec les conteneurs.
- **Volume externe** `sre` : la description du volume est lue auprès de `192.168.36.8:/sre` ; les briques sont hébergées hors du projet.
- Après lecture de la description, chaque client contacte directement les serveurs de briques du volume.
- **Stockage partagé** : une pièce jointe écrite par odoo1 devient accessible à odoo2 et odoo3.
- Le groupe `gluster_bricks` désigne les hôtes clients ; les hôtes PostgreSQL ne reçoivent pas ce montage.

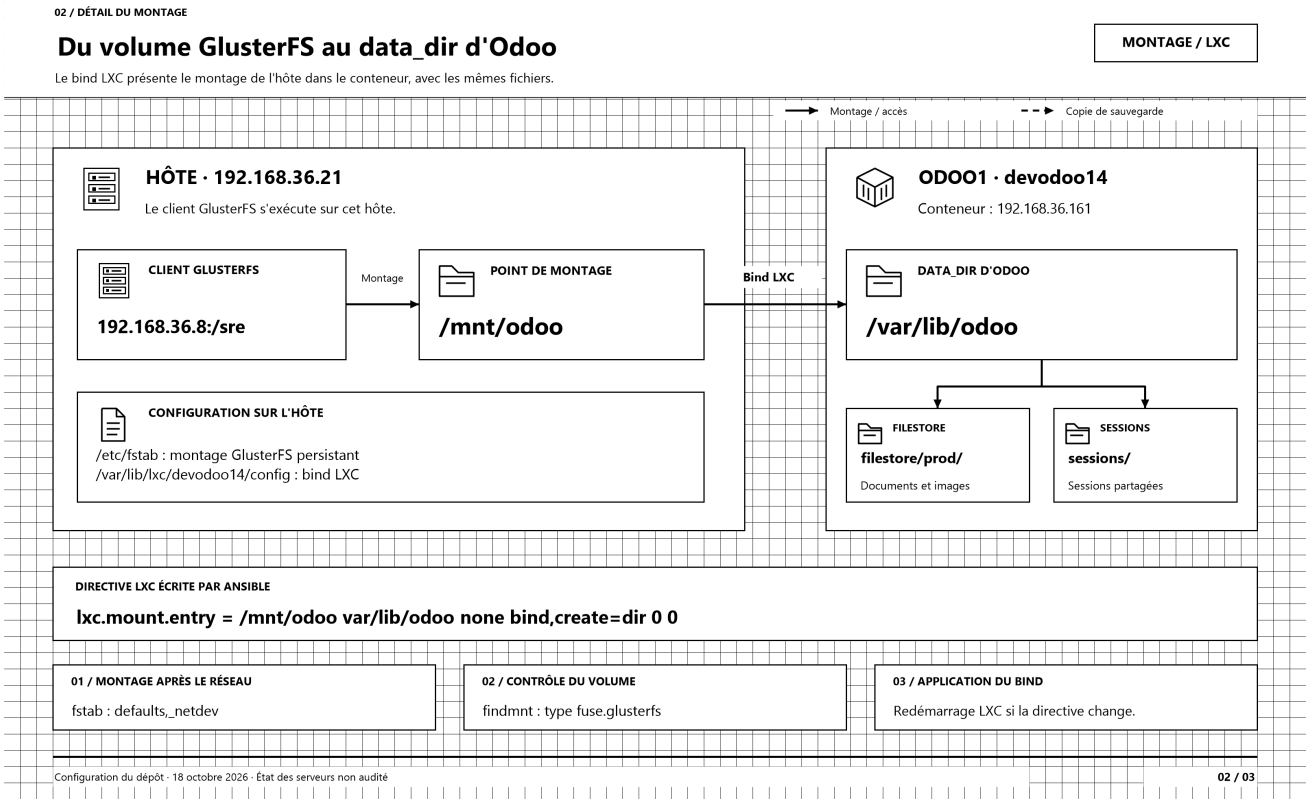


Schéma 16 – Ancien modèle : montage sur l'hôte et bind LXC

Lecture du schéma 16

Le volume monté sur l'hôte était passé au conteneur par un bind.

- Le client GlusterFS de l'hôte 192.168.36.21 montait `192.168.36.8:/sre` sur `/mnt/odoo`, de façon persistante par `/etc/fstab`.
- Un bind LXC présentait ce répertoire au conteneur odoo1 (192.168.36.161) sur `/var/lib/odoo`. Ansible écrivait dans `/var/lib/lxc/devodoo14/config` la directive `lxc.mount.entry = /mnt/odoo var/lib/odoo none bind,create=dir 0 0`.
- `/var/lib/odoo` est le `data_dir` d'Odoo : il contient `filestore/prod/` (documents et images) et `sessions/` (sessions partagées).
- Garde-fous de l'époque : montage après le réseau (`defaults,_netdev`), contrôle du type `fuse.glusterfs` par `findmnt`, redémarrage du conteneur si la directive changeait.

Pourquoi ce modèle a été abandonné : un hôte dont le montage échouait au démarrage passait au conteneur un `/mnt/odoo` vide. Le bind faisait de `/var/lib/odoo` un point de montage, la garde systemd laissait Odoo démarrer, et Odoo écrivait sur le disque local . Désormais, le conteneur monte lui-même le volume, et Odoo ne démarre que s'il est présent.

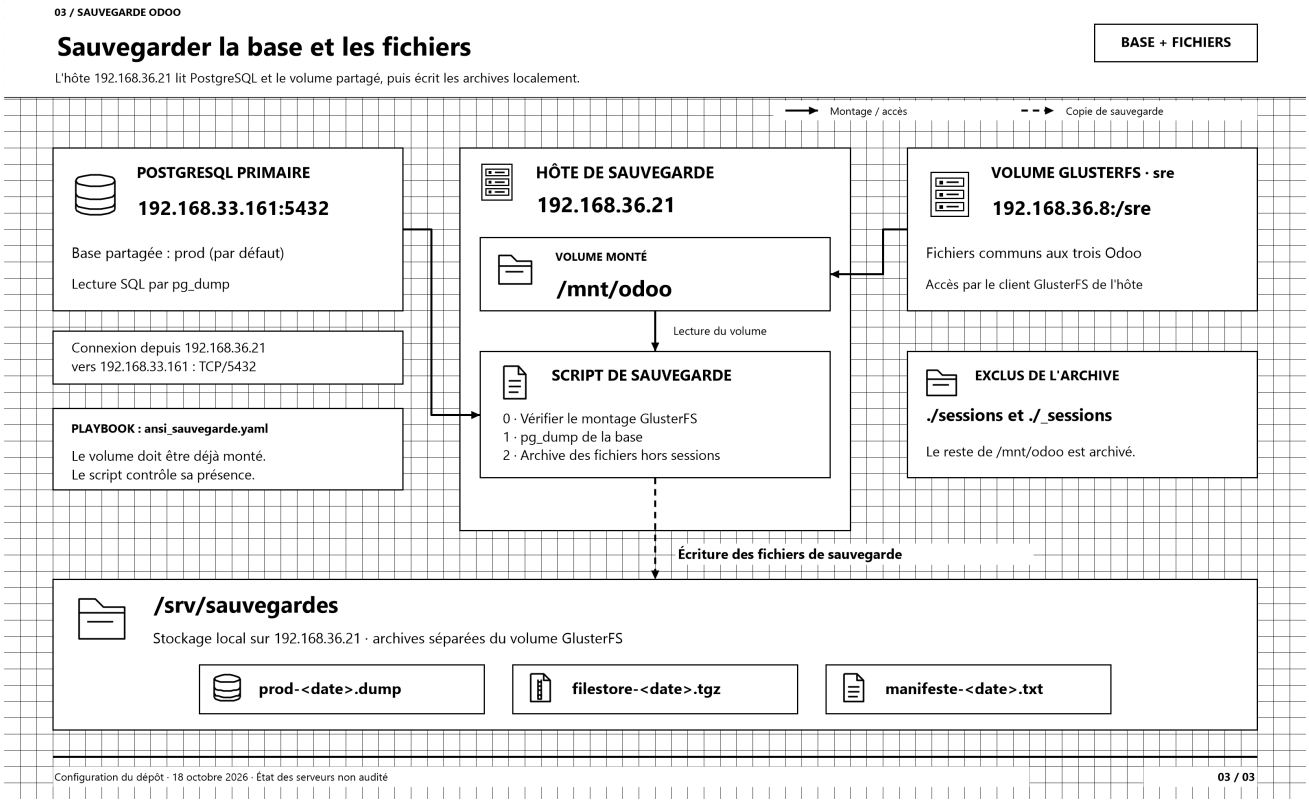


Schéma 17 – GlusterFS et sauvegarde

Lecture du schéma 17

La sauvegarde réunit un dump de la base et une archive du volume, sur un seul hôte.

- L'hôte de sauvegarde 192.168.36.21 lit le volume monté sur `/mnt/odoo`.
- Le script enchaîne trois étapes : vérifier le montage GlusterFS, faire un `pg_dump` de la base, puis archiver les fichiers. `./sessions` et `./_sessions` sont exclus ; le reste de `/mnt/odoo` est archivé.
- Base : primaire 192.168.33.161 :5432, base partagée `prod` par défaut, lue par `pg_dump` depuis 192.168.36.21.
- Les fichiers sont écrits dans `/srv/sauvegardes`, sur le disque local de .36.21, séparé du volume : `prod-<date>.dump`, `filestore-<date>.tgz` et `manifeste-<date>.txt`.
- Précondition : le volume doit être déjà monté ; le script contrôle sa présence. Le schéma cite le playbook `ansi_sauvegarde.yaml`, depuis supprimé du dépôt.

8.3 Architecture et HA PostgreSQL

Ces quatre vues datent d'avant l'activation de Patroni : la base était servie par `pg_server`, avec une bascule manuelle, et la haute disponibilité n'était encore qu'une option.

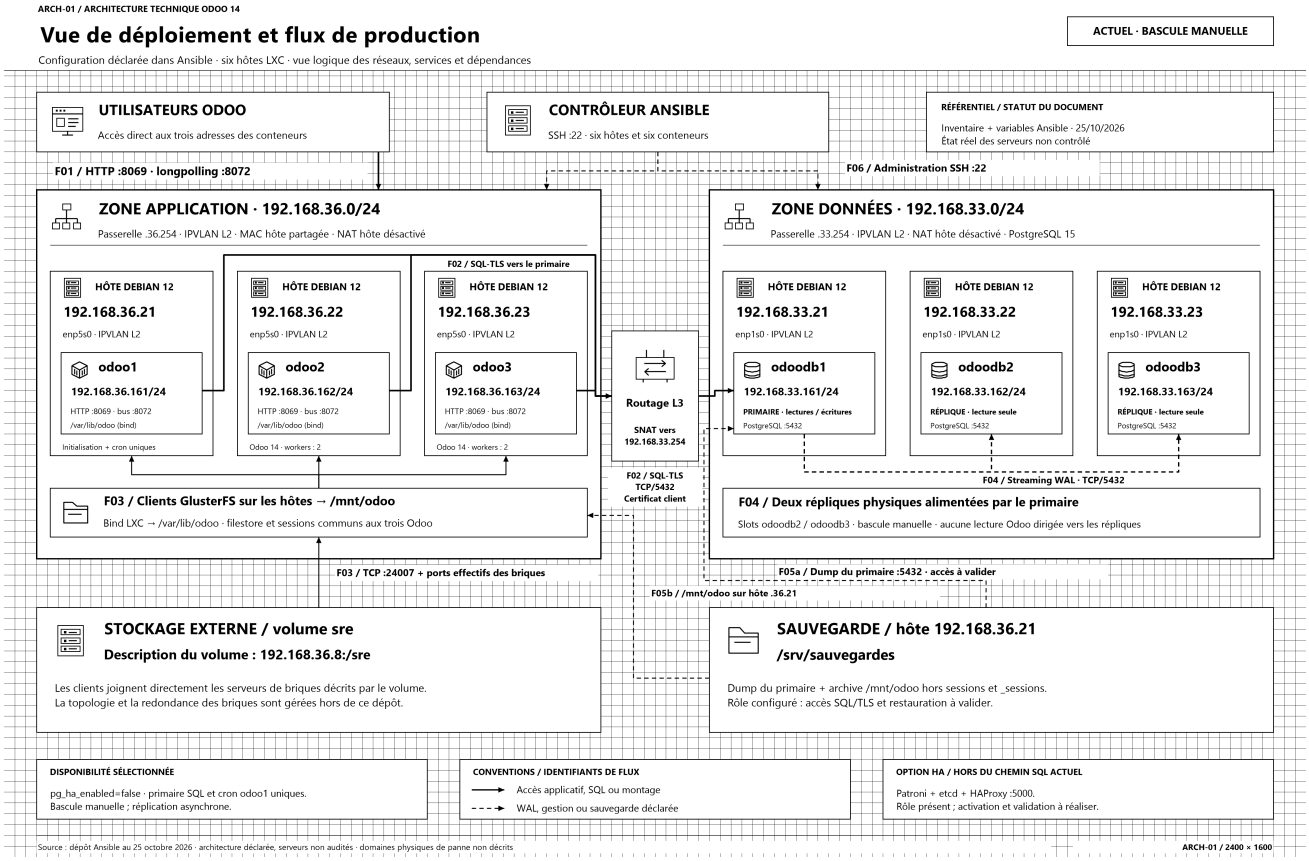


Schéma 18 – Vue de déploiement et flux de production (avant Patroni)

Lecture du schéma 18

Six hôtes LXC ; vue logique des réseaux, des services et des dépendances, flux par flux.

- Acteurs : les utilisateurs Odoo, qui accèdent directement aux trois conteneurs, et le contrôleur Ansible, en SSH sur 22 vers les six hôtes et les six conteneurs.
- Zones : application (192.168.36.0/24, passerelle .36.254) et données (192.168.33.0/24, passerelle .33.254), en IPVLAN L2 sans NAT sur les hôtes.
- **F01** HTTP 8069 et longpolling 8072 ; **F02** SQL-TLS vers le primaire sur 5432, avec certificat client, à travers le routage L3 et son SNAT vers .33.254 ; **F03** clients GlusterFS sur les hôtes vers `/mnt/odoo`, bind vers `/var/lib/odoo`, TCP/24007 et ports des briques ; **F04** streaming WAL vers deux répliques (slots `odoodb2` et `odoodb3`) ; **F05a** dump du primaire et **F05b** archive de `/mnt/odoo` sur .36.21 ; **F06** administration SSH.
- Disponibilité retenue à cette date : `pg_ha_enabled=false`, primaire SQL et cron odoo1 uniques, bascule manuelle, réplication asynchrone.
- Option HA hors du chemin SQL : Patroni, etcd et HAProxy sur 5000 ; rôle présent, activation et validation à réaliser.

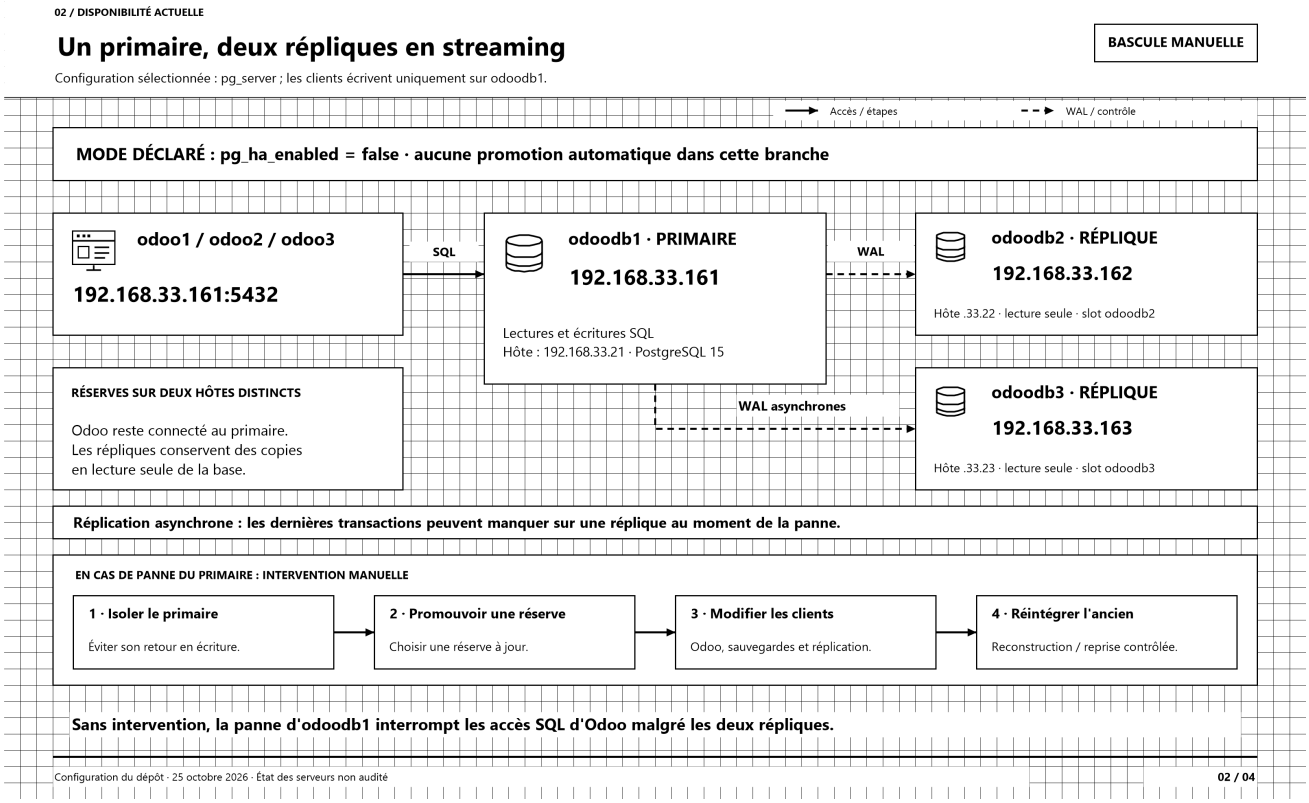


Schéma 19 – Réplication et bascule manuelle (avant Patroni)

Lecture du schéma 19

Un primaire, deux répliques en streaming ; les clients n'écrivent que sur odoodb1.

- Mode déclaré : `pg_ha_enabled = false`, rôle `pg_server`, aucune promotion automatique.
- odoo1, odoo2 et odoo3 se connectent tous à 192.168.33.161 :5432 : odoodb1, sur l'hôte .33.21, reçoit toutes les lectures et écritures SQL.
- odoodb2 (.33.162, hôte .33.22) et odoodb3 (.33.163, hôte .33.23) sont des répliques en lecture seule, alimentées par WAL asynchrones à travers les slots `odoodb2` et `odoodb3`.
- Les réserves vivent sur deux hôtes distincts et conservent des copies de la base. Les dernières transactions peuvent manquer sur une réplique au moment d'une panne.
- En cas de panne du primaire : intervention manuelle, c'est-à-dire promouvoir une réserve puis changer `odoo_db_host`. Sans intervention, la panne d'odoodb1 interrompt les accès SQL d'Odoo, malgré les deux répliques.

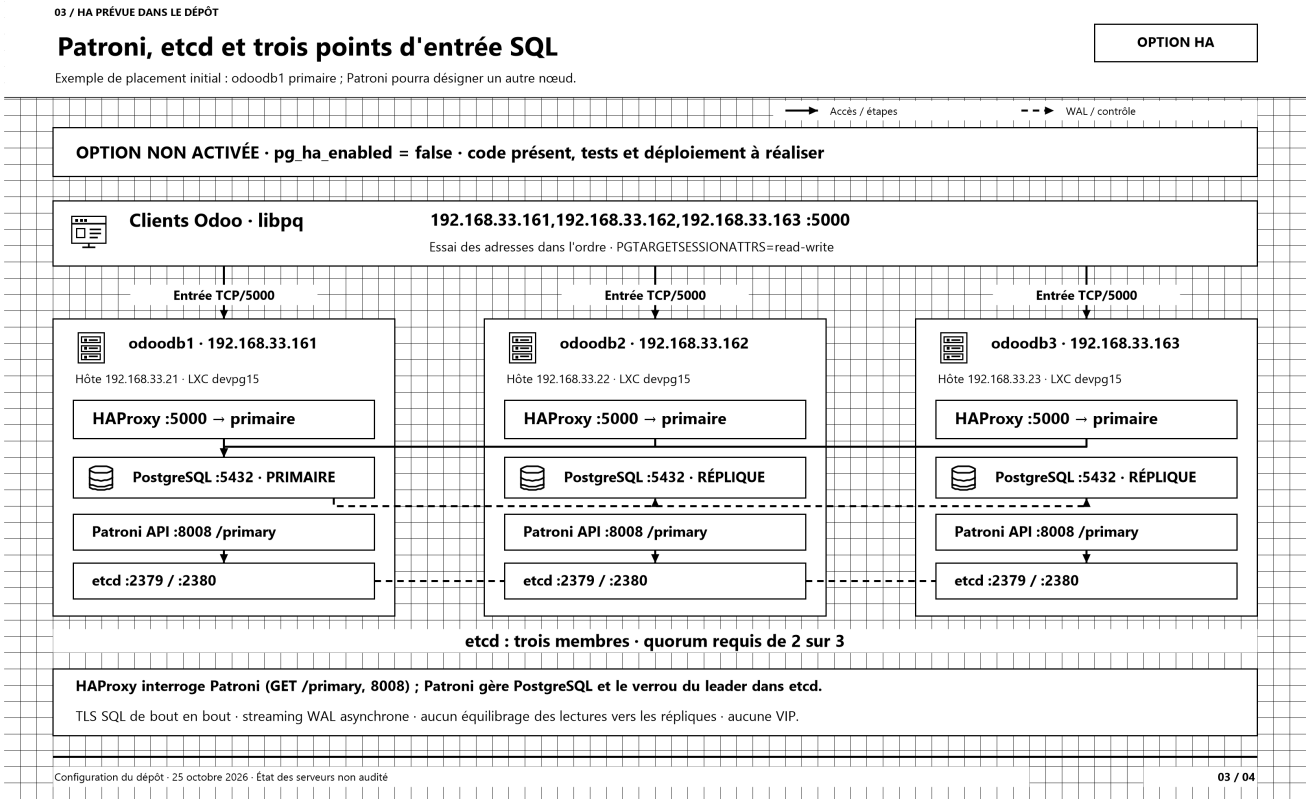


Schéma 20 – Option HA Patroni, etcd et HAProxy (telle que prévue)

Lecture du schéma 20

Patroni, etcd et trois points d'entrée SQL ; le placement initial n'est qu'un exemple.

- À cette date : option non activée (`pg_ha_enabled = false`) ; code présent, tests et déploiement à réaliser.
- Clients Odoo par libpq : `192.168.33.161,192.168.33.162,192.168.33.163` sur 5000, essayés dans l'ordre, avec `PGTARGETSESSIONATTRS=read-write`.
- etcd : trois membres, quorum requis de 2 sur 3.
- HAProxy interroge Patroni (`GET /primary`, 8008) ; Patroni gère PostgreSQL et le verrou du leader dans etcd.
- TLS SQL de bout en bout, streaming WAL asynchrone, aucun équilibrage des lectures vers les répliques, aucune adresse IP virtuelle.
- odoodb1 primaire est le placement initial ; Patroni peut désigner un autre nœud.

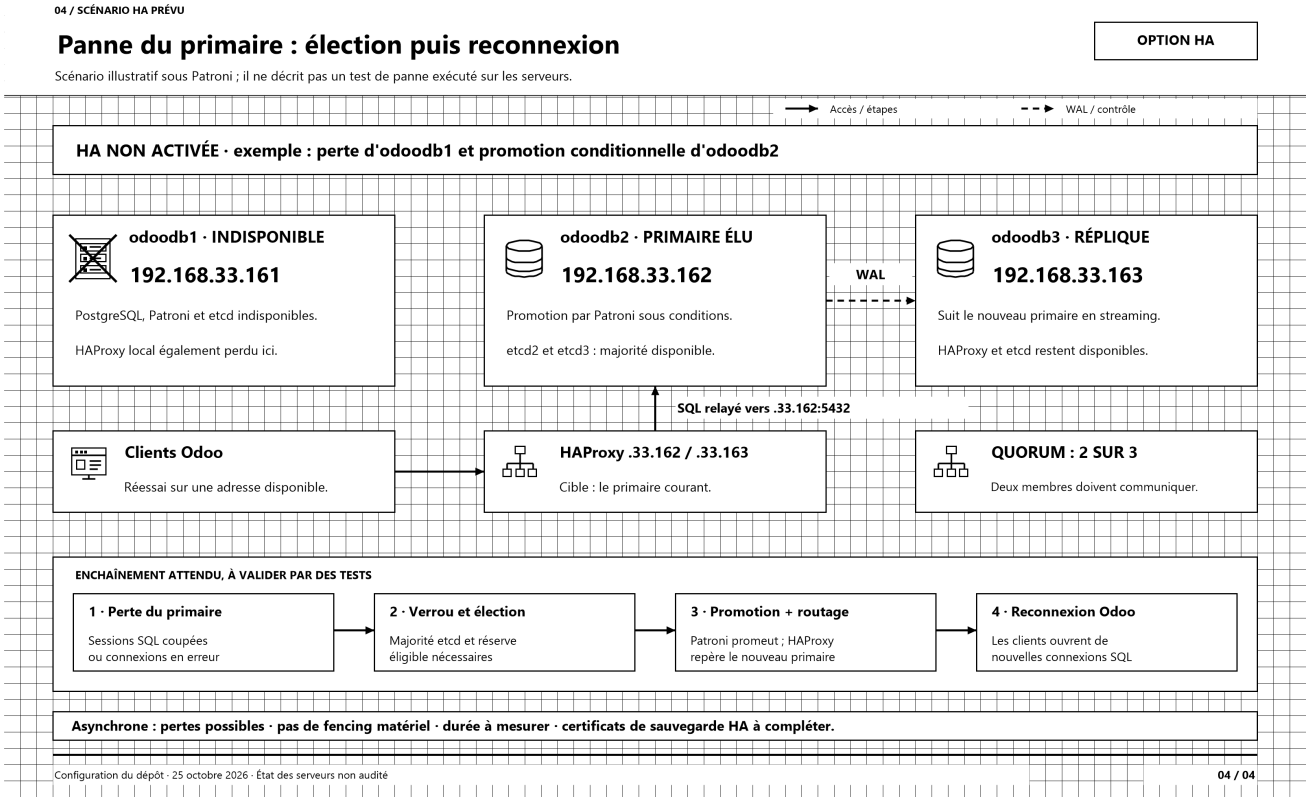


Schéma 21 – Bascule HA et limites de disponibilité

Lecture du schéma 21

Panne du primaire : élection, puis reconnexion. Scénario illustratif, non issu d'un test.

- odoodb1 (192.168.33.161) devient indisponible : PostgreSQL, Patroni, etcd et son HAProxy local sont perdus.
- odoodb2 (192.168.33.162) est élu primaire : promotion par Patroni sous conditions, etcd2 et etcd3 formant la majorité.
- odoodb3 (192.168.33.163) reste réplique et suit le nouveau primaire en streaming ; son HAProxy et son etcd restent disponibles.
- Les clients Odoo réessaient sur une adresse disponible ; les HAProxy .33.162 et .33.163 ciblent le primaire courant et relaient le SQL vers .33.162 :5432.
- Quorum de 2 sur 3 : deux membres doivent communiquer.
- Limites : pertes possibles en asynchrone, pas de fencing matériel, durée à mesurer, certificats de sauvegarde en HA à compléter.

9 Limites connues

- **Pas de chien de garde matériel** : un conteneur LXC n'a pas de `/dev/watchdog`. Un Patroni bloqué sans planter laisserait l'ancien primaire accepter des écritures pendant la promotion d'une réserve.
- **Réplication asynchrone** : lors d'une bascule non planifiée, des transactions récentes peuvent être perdues. Les transactions en cours ne sont pas transférées, et les clients doivent se reconnecter.

— **Quorum etcd de 2 sur 3** : la perte d'un serveur est tolérée. Si deux serveurs sont perdus, le dernier passe en lecture seule et Odoo s'arrête, par conception.

— **La haute disponibilité ne concerne que la base** : le cron d'Odoo reste sur odoo1, chaque accès web vise une instance, et le volume GlusterFS comme l'infrastructure physique sont hors du dépôt.

— **Groupes d'inventaire** : après une bascule, `db_containers` et `db_replicas` ne décrivent plus que le placement d'origine. `ansi_syncro.yaml` et `odoo_cleanup` traitent `db_containers` comme « la » base : il faut les vérifier avant de les lancer.

— **Fins de vie** : Odoo 14 n'est plus maintenu, et Python 3.8 ne reçoit plus de correctifs depuis octobre 2024. L'interpréteur est confiné à `/opt` et ne sert qu'à Odoo.

— **Sauvegarde incomplète en mode HA** : voir la section **??**.

— **Remise à zéro dangereuse** : `nettoyage_odoo.yaml` avec `full_reset=true` supprime `/var/lib/odoo`, devenu le point de montage du filestore partagé (section 6).

10 Conclusion

Le projet fournit trois instances Odoo 14 qui partagent une base unique et un filestore commun, avec une base PostgreSQL dont le primaire est remplacé automatiquement en cas de panne. Les chemins critiques sont protégés : TLS de bout en bout avec certificat client, autorités de certification séparées pour SQL et etcd, montage du filestore exigé avant tout démarrage d'Odoo, et initialisation unique de la base.

Trois chantiers restent prioritaires : rendre la sauvegarde opérationnelle en mode HA (playbook et certificat client), mesurer les délais réels de bascule et valider une restauration complète. Un répartiteur HTTP et la mobilité du cron d'Odoo étendraient ensuite la disponibilité à la couche applicative.

A Catalogue des schémas

Les schémas se trouvent dans le répertoire `docs/schemas-patroni-odoo14/` du dépôt. Chacun existe en PNG (4800 × 3200) pour la lecture, en SVG pour l'impression et en draw.io pour la modification. Le fichier `dossier-patroni-odoo14.drawio` les rassemble en quatorze onglets.

Nº	Schéma	Fichier (.png, .svg, .drawio)	Page
1	Architecture globale : Odoo et PostgreSQL HA	`01-architecture-globale-ha`	4
2	Déploiement : hôtes Debian et conteneurs LXC	`02-deploiement-hotes-lxc`	6
3	Patroni orchestre le rôle de chaque PostgreSQL	`03-orchestration-patroni`	7
4	Connexion Odoo : du venv au primaire courant	`04-connexion-odoo-haproxy`	8
5	Réplication : un primaire et deux répliques	`05-replication-postgresql`	10
6	Bascule automatique après panne du primaire	`06-bascule-automatique`	11
7	Quorum etcd et partitions réseau	`07-quorum-et-partitions`	12
8	Bascule planifiée pour maintenance	`08-bascule-planifiee`	13
9	Réintégrer un ancien primaire sous Patroni	`09-retour-ancien-primaire`	??
10	Transition : du cluster initial au mode Patroni	`10-transition-vers-patroni`	??
11	Flux réseau, certificats et surfaces de gestion	`11-flux-reseau-et-tls`	14
12	Sauvegarder puis restaurer Odoo et ses données	`12-sauvegarde-et-restauration`	??
13	Installer Odoo 14 sur Debian dans les LXC	`13-installation-odoo14-debian-lxc`	16
14	Pile d'exécution : LXC, venv et service Odoo	`14-pile-execution-odoo14-venv`	18

Tableau 9 – Catalogue des schémas de l'édition actuelle

Les schémas des éditions précédentes se trouvent dans trois autres répertoires de `docs/`, chacun avec son README, ses SVG et son fichier draw.io.

Nº	Schéma	Dossier de `docs/`, puis fichier (.png, .svg)	Page
15	Raccordement IPVLAN	`schemas-lxc/01-raccordement-ipvlan`	25
16	Accès utilisateur à Odoo	`schemas-lxc/02-acces-utilisateur-odoo`	26
17	Flux Odoo vers PostgreSQL	`schemas-lxc/03-flux-odoo-postgresql`	27
18	Volume GlusterFS partagé	`schemas-glusterfs/01-volume-partage-trois-odoo`	28
19	Montage sur l'hôte et bind LXC	`schemas-glusterfs/02-montage-hote-et-bind-lxc`	29
20	GlusterFS et sauvegarde	`schemas-glusterfs/03-glusterfs-et-sauvegarde`	30
21	Vue de déploiement et flux	`schemas-architecture-ha/01-architecture-actuelle`	31
22	Réplication et bascule manuelle	`schemas-architecture-ha/02-replication-et-bascule-manuelle`	32
23	Option HA Patroni, etcd, HAProxy	`schemas-architecture-ha/03-option-ha-patroni-etcd-haproxy`	33
24	Bascule HA et quorum	`schemas-architecture-ha/04-bascule-ha-et-quorum`	34

Tableau 10 – Catalogue des schémas des éditions précédentes